Bimodal Reference Manual

A Logic for Tense and Modality

Benjamin Brast-McKie
www.benbrastmckie.com
— July 26, 2026 —

Sources:

1. *"The Construction of Possible Worlds"*, Brast-McKie, *Journal of Philosophical Logic*, forthcoming.
2. The `ProofChecker` Lean 4 repository, `Theories/Bimodal/` – ground truth for all formal claims.

Abstract

This book presents the Bimodal logic **TM** for tense and modality, implemented and verified in the `ProofChecker` Lean 4 project. **TM** combines an S5 historical-necessity operator with linear temporal operators for past and future tense, axiomatized by the Burgess-Xu (BX) proof system over Until/Since primitives and interpreted over task-frame semantics. Soundness, the deduction theorem, the Lindenbaum lemma, and the perpetuity principles are fully proven; a canonical-model construction is developed for each frame class, and the completeness of **TM** with respect to its frame classes remains an open problem.

Part I (The Bimodal System) presents the formal system in full – syntax, semantics, proof theory, frame classes, metalogic, the operational decision procedure, and the derived-theorem library – and closes by positioning **TM** among richer temporal-modal logics (Vlach store/recall operators, the BL* tower, the decidability frontier). **Part II** (Applications) presents the proof-automation tactics, the dual-signal (proof-trace / countermodel) training-data pipeline, and worked dual-verification examples drawn directly from the Lean formalization.

Contents

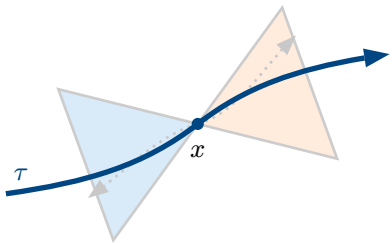
[1]The paper's Discreteness (DF), Density (DN), and Completeness (CO) frame properties, `possible_worlds.tex` §3.3 (:1162-1256) and appendices `app:discrete`/`app:dense`/`app:complete`.

1 Introduction

This book presents **TM**, a bimodal logic that unifies tense and modality in a single formally verified system, as implemented in the `ProofChecker` Lean 4 project. **TM** combines an S5 modal operator for historical necessity with linear temporal operators for past and future, axiomatized by the Burgess-Xu proof system over Until/Since primitives and interpreted over task-frame semantics. The semantic framework – the construction of possible worlds as histories over task frames – follows [1]; the philosophical background that motivates constructing possible worlds rather than positing them (the eternalism debate, the perpetuity principles as a touchstone, and the abundance dilemma) is developed there and is not rehearsed in this book. This book presents the bimodal system itself, in full: its formal specification (Part I) and its applications (Part II).

1.1 What TM Is

TM combines S5 historical modal operators for necessity ($\Box$) and possibility ($\Diamond$) with linear temporal operators for past (H) and future (G), all built over an **Until/Since** primitive basis. Precisely: **TM** is Until/Since temporal logic over linear orders ($\mathbb{Z}$ discrete, $\mathbb{Q}$ dense) fused with S5, plus a load-bearing modal-temporal interaction axiom (MF) and uniformity axiom layers over task frames. The Until/Since basis and the interaction axioms make **TM** a genuine fusion of temporal and modal reasoning rather than a temporal logic with a modal operator adjoined: the S5 modality quantifies across world histories, the temporal operators quantify within a single history, and the interaction axioms bind the two dimensions together (Chapter 15 discusses the relationship to the published presentation in detail). Beyond **TM** itself lies a natural extension hierarchy: **TM** $\rightarrow$ **TM**$^+$ (Vlach store/recall operators for cross-referencing traces and times) $\rightarrow$ the BL* tower, surveyed at the close of Part I.



*A single world history τ (the actual evolution, solid) through task-frame state space. From point x, the past/future light cones (shaded) contain the states that are modally accessible via $\Box/\Diamond$; the temporal operators H/G quantify strictly within a single history's past/future. Dotted paths are **not** alternative histories in **TM**'s formalization.*

The solid curve τ above represents a single world history – a temporal sequence of states. From any point x along a history, the past and future light cones contain all states that are modally accessible. The necessity operator $\Box$ quantifies over all possible histories, though we may often restrict to those histories that pass through the world state $\tau(x)$. The temporal operators H and G quantify over past and future times which, given a particular history, determine the range of past and future world states that history occupies relative to a given time. These primitive operators may then be used to define a host of combined operators of interest (Section 2.1).

1.2 Why Tense and Modality Together

Tense and modality interact, and the interaction is where the logical substance lies. The perpetuity principles – whatever is necessary was always and will always be necessary, and their converses and duals – are the touchstone: they are theses **about** the interaction of $\Box$ with H and G, and no treatment of either dimension in isolation can so much as state them. In **TM** the interaction is carried by a dedicated axiom (MF) together with the uniformity layers of the proof system, and the perpetuity principles P1–P6 are derived as theorems and machine-checked (Section 8.1). The fusion is therefore not a notational convenience: the metatheory of the combined system – its canonical models, its frame correspondences, its decision procedures – is substantially subtler than the metatheory of S5 and of Until/Since temporal logic taken separately, and Part I develops that metatheory in full.

A second motivation is verification. Every axiom, inference rule, and derived theorem presented in Part I resolves to a named declaration in the live Lean 4 source under `Theories/Bimodal/`, and the machine appendix at the end of the book lists the correspondence explicitly. This discipline pays for itself: formal statements in prose are easy to drift out of alignment with a developing formalization, whereas a book whose claims are checked against source admits a sharp distinction between what is proven, what is constructed but open, and what is paper-side mathematics. The book maintains that distinction throughout: proven results are cited by Lean name; open problems are stated as open problems; and results belonging to the companion papers rather than the formalization are attributed to the papers.

1.3 Outline

The book proceeds in two parts.

1. **Part I – The Bimodal System.** The full formal specification of **TM**: syntax, task-frame semantics, the Burgess-Xu proof system,

frame classes and their extensions, the metalogic, the operational decision procedure, and the derived-theorem library, closing with **TM**'s position among neighboring temporal-modal logics and the decidability frontier for its extensions.

2. **Part II – Applications.** Proof automation and the bounded proof-search engine, the dual-signal training-data pipeline (proof traces and countermodels, every output deterministically checkable), and dual-verification worked examples.

1.4 How to Read This Book

The parts are ordered by logical dependency, but several shorter paths through the material are available.

- **The core system.** The syntax, semantics, and proof-theory chapters form the spine of Part I; every later chapter presupposes them. A reader who wants only the definition of **TM** and its axiomatization can stop after the proof-theory chapter.
- **The metatheory.** The frame-classes, metalogic, and decidability chapters develop soundness, the canonical-model construction, and the tableau decision procedure. These chapters presuppose the spine but are independent of the derived-theorem library.
- **Comparative positioning.** The closing chapters of Part I – the LTL comparison, the Vlach/BL* survey, and the decidability frontier – locate **TM** among its neighbors and can be read independently after the spine.
- **Applications.** Part II is self-contained given the spine and the decidability chapter: proof automation, the training-data pipeline, and dual verification each occupy one chapter.

Formal claims are typeset with their Lean identifiers in fixed-width font (e.g. `perpetuity_1`); each such identifier names a declaration in the live source, and the machine appendix indexes the full correspondence.

1.5 Project Structure

The Lean 4 implementation is in the `Theories/Bimodal/` directory:

- `Syntax/` – Defines the formula language with 6 primitive constructors (atoms, $\bot$, implication, $\Box$, Until, Since) and derived operators.
- `ProofSystem/` – The Burgess-Xu (BX) axiom system: 42 axiom constructors in 8 layers and 7 inference rules forming a Hilbert-style proof system, parameterized by frame class (Base/Dense/Discrete).
- `Semantics/` – Task frames model possible worlds; world histories model time; strict (irreflexive) truth conditions define meaning.
- `FrameConditions/` – Frame-class semantics (dense, discrete) and per-class validity.

- `Metalogic/` – Soundness (proven for all three frame classes), deduction theorem and Lindenbaum lemma (proven), a canonical-model construction toward completeness (which remains an open problem), and a tableau-based decision procedure (soundness proven).
- `Theorems/` – Perpetuity principles (P1–P6, proven), modal and propositional theorem libraries, and derived temporal axioms.
- `Automation/`, `Examples/` – Proof tactics, the training-data pipeline, and worked examples, covered in Part II.

PART I

The Bimodal System

The formal specification of **TM**: syntax, task-frame semantics, the Burgess-Xu proof system, frame classes and their extensions, the metalogic, the operational decision procedure, and the derived-theorem library, followed by the system's position among neighboring temporal-modal logics and the decidability frontier for its extensions. The formal claims in this part resolve to live Lean source under `Theories/Bimodal/`.

2 Syntax

2.1 Formulas

Formulas are defined inductively with six primitive constructors, with **Until** and **Since** as the primitive temporal operators.

Definition 2.1 **Formula**

The type `Formula` is defined by:

$$\phi, \psi ::= p \mid \bot \mid \phi \to \psi \mid \Box\phi \mid U(\phi, \psi) \mid S(\phi, \psi)$$

where p ranges over sentence letters (type `Atom`).[2]

The binary temporal operators follow the **Burgess convention** [2] (`Syntax/Formula.lean`): in $U(\phi, \psi)$ the first argument ϕ is the **event**, true at some strictly future witness time, and the second argument ψ is the **guard**, true at all times strictly between now and the witness. $S(\phi, \psi)$ is the past mirror: the event held at some strictly past time, with the guard holding strictly in between.

Symbol	Name	Lean	Reading
p, q, r	Atom	`atom a`	sentence letter
$\bot$	Bottom	`bot`	falsity
$\phi \to \psi$	Implication	`imp`	"if ϕ, then ψ"
$\Box\phi$	Necessity	`box`	"necessarily ϕ"
$U(\phi, \psi)$	Until	`untl`	"ψ until ϕ"
$S(\phi, \psi)$	Since	`snce`	"ψ since ϕ"

The paper's base language $\mathcal{L}$ for **TM** instead takes the one-place tense operators H and G as primitive, introducing Until and Since only in its extended language $\mathcal{L}^+$. The Lean formalization works with the Until/Since basis throughout — a conservative change of presentation (the paper's **TM**$^+$ extends **TM** conservatively), under which H, G, F, and P become derived operators.

2.2 Derived Operators

The following operators are defined in terms of the primitives; each equation is the Lean `def` from `Syntax/Formula.lean`.

[2] `Atom` (`Syntax/Atom.lean`) pairs a base string with an optional freshness index, so that a fresh atom exists outside any finite set of atoms; `Formula.atom_s` builds an atom formula directly from a string.

Definition 2.2 **Propositional**

$$\top := \bot \to \bot$$
$$\neg\phi := \phi \to \bot$$
$$\phi \wedge \psi := \neg(\phi \to \neg\psi)$$
$$\phi \vee \psi := \neg\phi \to \psi$$

Symbol	Name	Lean	Reading
$\top$	Top	`top`	truth
$\neg\phi$	Negation	`neg`	"it is not the case that ϕ"
$\phi \wedge \psi$	Conjunction	`and`	"ϕ and ψ"
$\phi \vee \psi$	Disjunction	`or`	"ϕ or ψ"

Definition 2.3 **Modal**

$$\Diamond\phi := \neg\Box\neg\phi$$

Symbol	Name	Lean	Reading
$\Diamond\phi$	Possibility	`diamond`	"possibly ϕ"

Definition 2.4 **Temporal**

Following Burgess [2], the one-place tense operators are defined from Until and Since with a vacuous guard:

$$F\phi := U(\phi, \top)$$
$$P\phi := S(\phi, \top)$$
$$G\phi := \neg F\neg\phi$$
$$H\phi := \neg P\neg\phi$$
$$\triangle\,\phi := H\phi \wedge \phi \wedge G\phi$$
$$\triangledown\,\phi := P\phi \vee \phi \vee F\phi$$

Symbol	Name	Lean	Reading
$F\phi$	Sometime future	`some_future`	"it is going to be that ϕ"
$P\phi$	Sometime past	`some_past`	"it has been that ϕ"
$G\phi$	Always future	`all_future`	"it is always going to be that ϕ"
$H\phi$	Always past	`all_past`	"it has always been that ϕ"
$\triangle\,\phi$	Always	`always`	"always ϕ"
$\triangledown\,\phi$	Sometimes	`sometimes`	"sometimes ϕ"

Because F, P, G, and H are `def` abbreviations rather than constructors, they unfold definitionally; the semantics chapter gives their truth conditions as derived characterizations.

2.3 Temporal Duality

The `swap_temporal` function exchanges past and future operators.

Definition 2.5 **Temporal Swap**

The function $\langle S \rangle : \text{Formula} \to \text{Formula}$ is defined by recursion on the primitive constructors (`Syntax/Formula.lean`):

$$\begin{aligned} \langle S \rangle p &= p \\ \langle S \rangle \bot &= \bot \\ \langle S \rangle (\phi \to \psi) &= (\langle S \rangle \phi \to \langle S \rangle \psi) \\ \langle S \rangle \Box \phi &= \Box \langle S \rangle \phi \\ \langle S \rangle U(\phi, \psi) &= S(\langle S \rangle \phi, \langle S \rangle \psi) \\ \langle S \rangle S(\phi, \psi) &= U(\langle S \rangle \phi, \langle S \rangle \psi) \end{aligned}$$

On the derived operators this exchanges G with H and F with P.

Theorem 2.6 **Involution**

$\langle S \rangle \langle S \rangle \phi = \phi$[3]

3 Task Semantics

3.1 Task Frames

Task frames are the fundamental semantic structures for **TM**; the semantics presented in this chapter follows [1]. They abstract from universal laws governing transitions between world states while still retaining the temporal duration for a transition to complete.

The following primitives are required to define a task frame:

Primitive	Type	Description
W	Type	World states
D	Type	Temporal durations
$w \overset{x}{\Longrightarrow} u$	$W \to D \to W \to \text{Prop}$	Task relation

[3]Proven as `swap_temporal_involution` in `Syntax/Formula.lean`.

Definition 3.1 **Task Frame**

A **task frame** over temporal type D is a triple $\mathcal{F} = (W, D, \Longrightarrow)$ satisfying:

1. **Nullity**: For all $w : W$, we have $w \underset{0}{\Longrightarrow} w$.
2. **Reflection**: For all $w, u : W$ and $x : D$, if $w \underset{x}{\Longrightarrow} u$, then $u \underset{-x}{\Longrightarrow} w$.
3. **Compositionality**: For all $w, u, v : W$ and $x, y : D$, if $w \underset{x}{\Longrightarrow} u$ and $u \underset{y}{\Longrightarrow} v$, then $w \underset{x+y}{\Longrightarrow} v$.

Nullity ensures that zero-duration tasks leave the world state unchanged. Reflection ensures that every task is invertible by flipping the sign of its duration. Compositionality ensures that executing tasks sequentially yields results consistent with a single task of combined duration.

The Lean structure `TaskFrame` (`Semantics/TaskFrame.lean:93`) packages these constraints as three fields, in a form that is equivalent for all semantic purposes but algebraically tighter:

- `nullity_identity` : $w \underset{0}{\Longrightarrow} u \Leftrightarrow w = u$ — Nullity strengthened to an identity (zero duration means **no** change);
- `converse` : $w \underset{x}{\Longrightarrow} u \Leftrightarrow u \underset{-x}{\Longrightarrow} w$ — Reflection, in biconditional form;
- `forward_comp` : Compositionality restricted to non-negative durations $0 \leq x, y$ — the unrestricted mixed-sign form is algebraically impossible for non-deterministic relations, and the restricted form together with `converse` recovers every instance the semantics uses.

3.2 World Histories

A world history is a function from times to world states that respects the task relation over a convex temporal domain. World histories represent possible paths through the space of world states.

Definition 3.2 **Convex Domain**

A domain $\text{dom} : D \to \text{Prop}$ is **convex** if whenever $a, c \in \text{dom}$ with $a \leq c$, every time b with $a \leq b \leq c$ is also in dom. More precisely, for all $a, b, c : D$, if $\text{dom}(a)$ and $\text{dom}(c)$ and $a \leq b \leq c$, then $\text{dom}(b)$. Convexity ensures the domain has no temporal gaps.

Definition 3.3 **World History**

A **world history** in a task frame $\mathcal{F}$ is a dependent function $\tau : (x : D) \to \text{dom}(x) \to W$ where $\text{dom} : D \to \text{Prop}$ is a convex subset of D and $\tau(x) \underset{y-x}{\Longrightarrow} \tau(y)$ for all times $x, y : D$ with $\text{dom}(x)$, $\text{dom}(y)$, and $x \leq y$. We write $H_{\mathcal{F}}$ for all world histories over frame $\mathcal{F}$.

3.3 Task Models

A task model extends a task frame with an interpretation function that assigns truth values to sentence letters at world states. **Propositions** are subsets of W representing instantaneous ways for the system to be. Sentence letters express propositions, which can be realized by zero or more world states. World states themselves are specific configurations of the total system at an instant.

Definition 3.4 **Task Model**

A **task model** defined over a frame $\mathcal{F}$ is a pair $\mathcal{M} = (\mathcal{F}, I)$ where the **interpretation function** $I : W \to \text{Atom} \to \text{Prop}$ assigns to each world state $w : W$ and sentence letter $p : \text{Atom}$ a truth value $I(w, p) : \text{Prop}$. We write $I(w, p)$ to indicate that sentence letter p is true at w.

3.4 Truth Conditions

Truth is evaluated relative to a model $\mathcal{M}$ providing the interpretation, a world history τ representing a possible path through the space of world states, and a time $x : D$. Whereas the model fixes the interpretation of the language, the contextual parameters τ and x determine the truth value of every sentence of the language.

Definition 3.5 **Truth**

For model $\mathcal{M}$, history $\tau : H_{\mathcal{F}}$, and time $x : D$, truth is defined by recursion on the six primitive constructors:[4]

$$\mathcal{M}, \tau, x \vDash p \;\; iff \;\; x \in \text{dom}(\tau) \text{ and } I(\tau(x), p)$$

$$\mathcal{M}, \tau, x \nvDash \bot$$

$$\mathcal{M}, \tau, x \vDash \phi \to \psi \;\; iff \;\; \mathcal{M}, \tau, x \nvDash \phi \text{ or } \mathcal{M}, \tau, x \vDash \psi$$

$$\mathcal{M}, \tau, x \vDash \Box\phi \;\; iff \;\; \mathcal{M}, \sigma, x \vDash \phi \text{ for all } \sigma : H_{\mathcal{F}}$$

$$\mathcal{M}, \tau, x \vDash U(\phi, \psi) \;\; iff \text{ for some } y : D \text{ with } x < y \text{ we have } \mathcal{M}, \tau, y \vDash \phi \text{ and } \mathcal{M}, \tau, z \vDash \psi \text{ for all } z : D \text{ where } x < z < y$$

$$\mathcal{M}, \tau, x \vDash S(\phi, \psi) \;\; iff \text{ for some } y : D \text{ with } y < x \text{ we have } \mathcal{M}, \tau, y \vDash \phi \text{ and } \mathcal{M}, \tau, z \vDash \psi \text{ for all } z : D \text{ where } y < z < x$$

Until and Since use a **strict witness** with an **open guard**: the witness time y is strictly future (respectively strictly past), and the guard ψ is required only on the open interval strictly between x and y. The derived tense operators then receive their expected **strict** truth conditions as

[4] `truth_at` in `Semantics/Truth.lean`.

characterization theorems:[5]

Theorem 3.6 **Derived Truth Conditions**

$$\mathcal{M}, \tau, x \vDash H\phi \ iff \ \mathcal{M}, \tau, y \vDash \phi \text{ for all } y : D \text{ where } y < x$$
$$\mathcal{M}, \tau, x \vDash G\phi \ iff \ \mathcal{M}, \tau, y \vDash \phi \text{ for all } y : D \text{ where } x < y$$
$$\mathcal{M}, \tau, x \vDash P\phi \ iff \ \mathcal{M}, \tau, y \vDash \phi \text{ for some } y : D \text{ where } y < x$$
$$\mathcal{M}, \tau, x \vDash F\phi \ iff \ \mathcal{M}, \tau, y \vDash \phi \text{ for some } y : D \text{ where } x < y$$

The modal operator $\Box$ quantifies over all world histories $\sigma : H_{\mathcal{F}}$ at the current time $x : D$. The temporal operators quantify over all earlier and later times $y : D$ — over the whole temporal order, not merely the history's domain, so atoms are false (rather than undefined) at times outside $\mathrm{dom}(\tau)$.

3.5 Time-Shift

The time-shift operation is used to establish the **perpetuity principles**:

- P1: $\Box\phi \to \triangle\,\phi$ (what is necessary is always the case)
- P2: $\triangledown\,\phi \to \Diamond\phi$ (what is sometimes the case is possible)

It is natural to assume that whatever is necessary is always the case, or equivalently, whatever is sometimes the case is possible. Time-shift enables the validity proof of the bimodal interaction axiom MF ($\Box\phi \to \Box G\phi$); together with the derived theorem TF ($\Box\phi \to G\Box\phi$) it yields the perpetuity principles.

Definition 3.7 **Time-Shift**

For $\tau, \sigma \in H_{\mathcal{F}}$ and $x, y : D$, world histories τ and σ are **time-shifted from y to x**, written $\tau \overset{x}{\underset{y}{\approx}} \sigma$, if and only if there exists an order automorphism $\overline{a} : D \to D$ where $y = \overline{a}(x)$, $\mathrm{dom}_\sigma = \overline{a}^{-1}(\mathrm{dom}_\tau)$, and $\sigma(z) = \tau(\overline{a}(z))$ for all $z \in \mathrm{dom}_\sigma$.

Time-shifting preserves the essential structure of histories:

Theorem 3.8 **Convexity Preservation**

If τ has a convex domain and $\tau \overset{x}{\underset{y}{\approx}} \sigma$, then σ has a convex domain.

Theorem 3.9 **Task Preservation**

If τ respects the task relation and $\tau \overset{x}{\underset{y}{\approx}} \sigma$, then σ respects the task relation.

[5] `future_iff`, `past_iff`, and companions in `Semantics/Truth.lean`; the semantics is irreflexive: G and H exclude the present moment, so the temporal T-axioms $G\phi \to \phi$ and $H\phi \to \phi$ are not valid.

3.6 Logical Consequence and Validity

Logical consequence and validity are defined uniformly across all temporal types, frames, models, histories, and times.

Definition 3.10 **Logical Consequence**

A formula ϕ is a **logical consequence** of Γ (written $\Gamma \vDash \phi$) just in case for every temporal type $D : \text{Type}$, frame $\mathcal{F} : \text{TaskFrame}(D)$, model $\mathcal{M} : \text{TaskModel}(\mathcal{F})$, history $\tau \in H_{\mathcal{F}}$, and time $x : D$, if $\mathcal{M}, \tau, x \vDash \psi$ for all $\psi \in \Gamma$, then $\mathcal{M}, \tau, x \vDash \phi$.

Definition 3.11 **Validity**

A formula ϕ is **valid** (written $\vDash \phi$) just in case ϕ is a logical consequence of the empty set: $\emptyset \vDash \phi$. Equivalently, ϕ is true at every model-history-time triple.

Definition 3.12 **Satisfiability**

A context Γ is **satisfiable** in temporal type $D : \text{Type}$ if there exist a frame $\mathcal{F} : \text{TaskFrame}(D)$, model $\mathcal{M} : \text{TaskModel}(\mathcal{F})$, history $\tau \in H_{\mathcal{F}}$, and time $x : D$ such that $\mathcal{M}, \tau, x \vDash \psi$ for all $\psi \in \Gamma$.

Theorem 3.13 **Monotonicity**

If $\Gamma \subseteq \Delta$ and $\Gamma \vDash \phi$, then $\Delta \vDash \phi$.

Remark **Determined and Deterministic**

The task-frame setting supports a distinction between a history's future being **determined** (every history through the present world state agrees on it) and the frame being **deterministic** (the task relation is functional), developed together with actuality operators in [1]. Neither notion has a formal Lean counterpart in this repository; both belong to the semantic framework's paper-side development.

4 Proof Theory

The Lean proof system for **TM** is the **Burgess-Xu (BX) axiom system**: a Hilbert-style calculus over the Until/Since language of Section 2.1, with **42 axiom constructors** organized into eight layers and **7 inference rules**. Derivations are parameterized by a **frame class** (`Base`, `Dense`, or `Discrete`), which gates the frame-dependent axiom layers. The system is deliberately more fine-grained than the economical 12-schema presentation of **TM** in [1]; the correspondence is explained in Section 4.6.

4.1 The Burgess-Xu Axiom System

The 42 axiom schemata are the constructors of the inductive family `Axiom` in `ProofSystem/Axioms.lean`. Throughout, $U(\phi, \psi)$ and $S(\phi, \psi)$ follow the **Burgess convention** used by the Lean constructors `until`/`snce`: the first argument is the **event** (true at the witness time) and the second is the **guard** (true at all strictly intermediate times); see Section 2.1. Derived operators: $F\phi = U(\phi, \top)$, $P\phi = S(\phi, \top)$, $G\phi = \neg F \neg \phi$, $H\phi = \neg P \neg \phi$, and $\top = \bot \to \bot$.

4.1.1 Layer 1: Propositional (4)

Name	Lean Constructor	Schema
K	`Axiom.prop_k`	$(\phi \to (\psi \to \chi)) \to ((\phi \to \psi) \to (\phi \to \chi))$
S	`Axiom.prop_s`	$\phi \to (\psi \to \phi)$
EFQ	`Axiom.ex_falso`	$\bot \to \phi$
Peirce	`Axiom.peirce`	$((\phi \to \psi) \to \phi) \to \phi$

Together with modus ponens, these four schemata axiomatize classical propositional logic (K and S give the implicational fragment; EFQ and Peirce restore classical negation over the primitive $\bot$).

4.1.2 Layer 2: S5 Modal (5)

Name	Lean Constructor	Schema
MT	`Axiom.modal_t`	$\Box\phi \to \phi$
M4	`Axiom.modal_4`	$\Box\phi \to \Box\Box\phi$
MB	`Axiom.modal_b`	$\phi \to \Box\Diamond\phi$
M5	`Axiom.modal_5_collapse`	$\Diamond\Box\phi \to \Box\phi$
MK	`Axiom.modal_k_dist`	$\Box(\phi \to \psi) \to (\Box\phi \to \Box\psi)$

The metaphysical necessity operator $\Box$ is S5: it quantifies over all admissible world histories at the current time.

4.1.3 Layer 3: BX Temporal (22)

The temporal core consists of eleven schemata in future/past mirror pairs, following Burgess [2], [3] and Xu [4] for Until/Since logic on linear orders. The primed names denote past mirrors.

Name	Lean Constructor	Schema
BX1	`Axiom.serial_future`	$\top \to F\top$
BX1′	`Axiom.serial_past`	$\top \to P\top$
BX2G	`Axiom.left_mono_until_G`	$G(\phi \to \chi) \to (U(\psi, \phi) \to U(\psi, \chi))$
BX2H	`Axiom.left_mono_since_H`	$H(\phi \to \chi) \to (S(\psi, \phi) \to S(\psi, \chi))$
BX3	`Axiom.right_mono_until`	$G(\phi \to \psi) \to (U(\phi, \chi) \to U(\psi, \chi))$
BX3′	`Axiom.right_mono_since`	$H(\phi \to \psi) \to (S(\phi, \chi) \to S(\psi, \chi))$
BX4	`Axiom.connect_future`	$\phi \to GP\phi$
BX4′	`Axiom.connect_past`	$\phi \to HF\phi$
BX5	`Axiom.self_accum_until`	$U(\psi, \phi) \to U(\psi, \phi \wedge U(\psi, \phi))$
BX5′	`Axiom.self_accum_since`	$S(\psi, \phi) \to S(\psi, \phi \wedge S(\psi, \phi))$
BX6	`Axiom.absorb_until`	$U(\phi \wedge U(\psi, \phi), \phi) \to U(\psi, \phi)$
BX6′	`Axiom.absorb_since`	$S(\phi \wedge S(\psi, \phi), \phi) \to S(\psi, \phi)$
BX7	`Axiom.linear_until`	$U(\psi, \phi) \wedge U(\theta, \chi) \to U(\psi \wedge \theta, \phi \wedge \chi) \vee U(\psi \wedge \chi, \phi \wedge \chi) \vee U(\phi \wedge \theta, \phi \wedge \chi)$
BX7′	`Axiom.linear_since`	$S(\psi, \phi) \wedge S(\theta, \chi) \to S(\psi \wedge \theta, \phi \wedge \chi) \vee S(\psi \wedge \chi, \phi \wedge \chi) \vee S(\phi \wedge \theta, \phi \wedge \chi)$
BX10	`Axiom.until_F`	$U(\psi, \phi) \to F\psi$
BX10′	`Axiom.since_P`	$S(\psi, \phi) \to P\psi$
BX11	`Axiom.temp_linearity`	$F\phi \wedge F\psi \to F(\phi \wedge \psi) \vee F(\phi \wedge F\psi) \vee F(F\phi \wedge \psi)$
BX11′	`Axiom.temp_linearity_past`	$P\phi \wedge P\psi \to P(\phi \wedge \psi) \vee P(\phi \wedge P\psi) \vee P(P\phi \wedge \psi)$
BX12	`Axiom.F_until_equiv`	$F\phi \to U(\phi, \top)$
BX12′	`Axiom.P_since_equiv`	$P\phi \to S(\phi, \top)$
BX13	`Axiom.enrichment_until`	$p \wedge U(\psi, \phi) \to U(\psi \wedge S(p, \phi), \phi)$
BX13′	`Axiom.enrichment_since`	$p \wedge S(\psi, \phi) \to S(\psi \wedge U(p, \phi), \phi)$

Table 8: BX temporal layer. Gaps in the numbering (BX2, BX8, BX9, BX14) mark schemata of Burgess [2] that were removed as unsound or unnecessary under the strict-witness/open-guard semantics; see the source comments in `ProofSystem/Axioms.lean`.

Highlights of the layer:

- **Seriality** (BX1/BX1′) replaces the temporal T-axioms, which are invalid under the strict semantics: every time has a strictly later and a strictly earlier time.
- **Monotonicity** (BX2G/BX2H, BX3/BX3′) lets Until and Since respect implications holding over the guard interval and at the event.
- **Connectedness** (BX4/BX4′) places the present in the past of every future time and in the future of every past time.
- **Self-accumulation and absorption** (BX5/BX6 and mirrors) resolve Until-eventualities axiomatically: an eventuality enriches its own guard, and deferred eventualities collapse.
- **Linearity** (BX7, BX11 and mirrors) orders witnesses linearly, as required on linear temporal orders.
- **Eventuality bridges** (BX10, BX12 and mirrors) connect Until/Since to the derived F/P operators.
- **Enrichment** (BX13/BX13′, Burgess A3a/A3b [2]) carries information about the current point into the event of an Until/Since formula.

4.1.4 Layer 4: Modal-Temporal Interaction (1)

Name	Lean Constructor	Schema
MF	`Axiom.modal_future`	$\Box\phi \to \Box G\phi$

MF is the sole bimodal interaction axiom: necessary truths remain necessary in the future. Its companion TF ($\Box\phi \to G\Box\phi$) is **derived** (see Section 4.3).

4.1.5 Layer 5: Uniformity (5)

The uniformity axioms concern the discreteness witness $U(\top, \bot)$ ("there is an immediate successor": the guard interval to the witness is empty). They encode the **uniformity of discreteness** in ordered abelian groups — by translation invariance, a gap at one point exists at every point and in every accessible world — and are therefore valid on **all** frames (frame class `Base`).

Lean Constructor	Schema
`Axiom.discrete_symm_fwd`	$U(\top, \bot) \to S(\top, \bot)$
`Axiom.discrete_symm_bwd`	$S(\top, \bot) \to U(\top, \bot)$
`Axiom.discrete_propagate_fwd`	$U(\top, \bot) \to G(U(\top, \bot))$
`Axiom.discrete_propagate_bwd`	$U(\top, \bot) \to H(U(\top, \bot))$
`Axiom.discrete_box_necessity`	$U(\top, \bot) \to \Box(U(\top, \bot))$

4.1.6 Layers 6–7: Prior and Z1 (3, discrete-only)

Valid on discrete linear orders (frame class `Discrete`), these axioms encode well-ordering for definable sets.

Name	Lean Constructor	Schema
Prior-UZ	`Axiom.prior_UZ`	$F\phi \to U(\phi, \neg\phi)$
Prior-SZ	`Axiom.prior_SZ`	$P\phi \to S(\phi, \neg\phi)$
Z1	`Axiom.z1`	$G(G\phi \to \phi) \to (FG\phi \to G\phi)$

Prior-UZ says that if ϕ holds somewhere in the future, there is a **nearest** future ϕ-point [5] (Venema's axiom (W)); Z1 is the characteristic backward-induction axiom of successor-Archimedean orders such as $\mathbb{Z}$ [6].

4.1.7 Layer 8: Density (2, dense-only)

Valid on densely ordered frames (frame class `Dense`).

Name	Lean Constructor	Schema
DN	`Axiom.density`	$GG\phi \to G\phi$
DI	`Axiom.dense_indicator`	$\neg U(\top, \bot)$

On a dense order no point has an immediate successor, so the discreteness witness $U(\top, \bot)$ is false everywhere (DI, the Burgess density axiom for Until/Since [2]). DI is included alongside DN because the density schema alone provably fails to derive it.

4.2 Frame Classes

Derivations are parameterized by a frame class, making frame-dependent reasoning a structural invariant rather than a side condition.

Definition 4.1 **Frame Class**

The type `FrameClass` has three values: `Base`, `Dense`, and `Discrete`, partially ordered with `Base` below both `Dense` and `Discrete`, which are incomparable. Each axiom constructor is assigned a minimum frame class by `Axiom.minFrameClass`: the 37 axioms of layers 1–5 are `Base`; Prior-UZ, Prior-SZ, and Z1 (3 axioms) are `Discrete`; DN and DI (2 axioms) are `Dense`.

The axiom rule of the proof system admits an axiom into a derivation at frame class `fc` only when its minimum frame class is at most `fc`. Derivations are monotone along the order: `DerivationTree.lift` coerces a derivation at `Base` into one at `Dense` or `Discrete`.

This mirrors the paper's hierarchy of **TM** extensions ([1] §3.3): frame class `Base` corresponds to the base system ($\mathbf{TM}^+$, the paper's Until/Since extension of **TM**, of which **TM** is a conservative reduct —

see `Metalogic/ConservativeExtension/`), `Dense` to the dense extension $\mathbf{TM}_{\mathrm{d}}$, and `Discrete` to the discrete extension $\mathbf{TM}_{\mathrm{f}}$. The paper's complete extension $\mathbf{TM}_{\mathrm{c}}$ (order-completeness) and combined $\mathbf{TM}_{\mathrm{dc}}$ are paper-side extensions and do not correspond to frame classes in the Lean formalization.

4.3 Derived Axioms

Several schemata that are primitive axioms of the paper's **TM** presentation are **derived theorems** of BX:

Name	Schema	Derived As
TK	$G(\phi \to \psi) \to (G\phi \to G\psi)$	`temp_k_dist_derived` (`Theorems/TemporalDerived.lean`)
T4	$G\phi \to GG\phi$	`temp_4_derived` (`Theorems/TemporalDerived.lean`)
TF	$\Box\phi \to G\Box\phi$	`temp_future_derived` (`Theorems/Combinators.lean`)

TK and T4 follow from the BX Until/Since axioms; TF follows from MF together with MT and M4. Two further schemata of the paper's presentation are BX axioms under different names: the paper's TB ($F\top$, seriality) is BX1 `serial_future`, and the paper's TA ($\phi \to GP\phi$) is BX4 `connect_future`. The paper's TL (future linearity) is BX11 `temp_linearity`.

4.4 Inference Rules

The proof system has 7 inference rules, given as the constructors of `DerivationTree`. Here $\Gamma \underset{fc}{\vdash} \phi$ abbreviates `DerivationTree fc Γ φ`; the plain turnstile $\Gamma \vdash \phi$ fixes the frame class to `Base`.

Definition 4.2 **Axiom Rule**

If ϕ matches an axiom schema whose minimum frame class is at most fc, then $\Gamma \underset{fc}{\vdash} \phi$.

Definition 4.3 **Assumption Rule**

If $\phi \in \Gamma$, then $\Gamma \underset{fc}{\vdash} \phi$.

Definition 4.4 **Modus Ponens**

$$\frac{\Gamma \underset{fc}{\vdash} \phi \to \psi \qquad \Gamma \underset{fc}{\vdash} \phi}{\Gamma \underset{fc}{\vdash} \psi}$$

Definition 4.5 **Necessitation**

$$\frac{\underset{fc}{\vdash} \phi}{\underset{fc}{\vdash} \Box\phi}$$

Applies only to theorems (empty context).

Definition 4.6 **Temporal Necessitation**

$$\frac{\underset{fc}{\vdash} \phi}{\underset{fc}{\vdash} G\phi}$$

Applies only to theorems (empty context).

Definition 4.7 **Temporal Duality**

$$\frac{\underset{fc}{\vdash} \phi}{\underset{fc}{\vdash} \langle S \rangle \phi}$$

Applies only to theorems (empty context).

Definition 4.8 **Weakening**

$$\frac{\Gamma \underset{fc}{\vdash} \phi \qquad \Gamma \subseteq \Delta}{\Delta \underset{fc}{\vdash} \phi}$$

Rule	Lean Constructor	Context Requirement
Axiom	`DerivationTree.axiom`	Any (gated by `minFrameClass`)
Assumption	`DerivationTree.assumption`	Any
Modus Ponens	`DerivationTree.modus_ponens`	Any
Necessitation	`DerivationTree.necessitation`	Empty only
Temp. Necessitation	`DerivationTree.temporal_necessitation`	Empty only
Temporal Duality	`DerivationTree.temporal_duality`	Empty only
Weakening	`DerivationTree.weakening`	Any

4.5 Derivation Trees

Derivations are represented as inductive trees.

Definition 4.9 **Derivation Tree**

`DerivationTree fc Γ φ` (written $\Gamma \underset{fc}{\vdash} \phi$) is an inductive type representing a derivation of ϕ from context Γ at frame class fc, using only axioms whose minimum frame class is at most fc.

Definition 4.10 **Height**

The height of a derivation tree:
- Base cases (axiom, assumption): height 0
- Unary rules: height of subderivation + 1
- Modus ponens: max of both subderivations + 1

`DerivationTree` is a `Type` (not a `Prop`), so derivations can be pattern-matched and measured; the height function enables well-founded recursion in metalogical proofs (notably the deduction theorem).

4.6 Relation to the Paper's Presentation

The paper presents **TM** economically, as the smallest extension of classical propositional logic closed under twelve schemata: the rules MP, MN, and TD, and the axioms MK, MT, M5, MF, TK, T4, TB, TA, and TL (with H/G primitive in the base language $\mathcal{L}$). The Lean system is the constructor-level **BX** axiomatization of the paper's $\mathbf{TM}^+$ (the Until/Since system of the paper's §3.3, proven there to extend **TM** conservatively). The differences are presentational granularity, not logical strength:

- **CPL is spelled out**: the paper subsumes classical propositional logic in a single phrase; Lean lists its four Hilbert schemata (Layer 1) explicitly.
- **S5 is closed under theorems**: Lean includes M4 and MB alongside the paper's MT, M5, MK — derivable in S5 but convenient as primitives.
- **Until/Since replaces H/G as the temporal engine**: with U/S primitive, the paper's TK and T4 become derived theorems, TB and TA reappear as BX1 and BX4, and TL is BX11 (Section 4.3).
- **Past mirrors are primed constructors**: the paper generates past duals by the TD rule; Lean additionally includes primed mirror constructors (BX1′–BX13′), which makes derivations at non-empty contexts more direct.
- **Frame-class axioms are gated structurally**: the paper's extensions $\mathbf{TM}_{\mathrm{f}}$ and $\mathbf{TM}_{\mathrm{d}}$ become the `Discrete` and `Dense` frame classes of Section 4.2 rather than separate systems.

4.7 Notation

Notation	Lean
$\Gamma \vdash \phi$	`Γ ⊢ φ` i.e. `DerivationTree FrameClass.Base Γ φ`
$\Gamma \underset{fc}{\vdash} \phi$	`Γ ⊢[fc] φ` i.e. `DerivationTree fc Γ φ`
$\vdash \phi$	`DerivationTree FrameClass.Base [] φ`

5 Frame Classes and Extensions

How the base proof system of Section 2.1 and the proof theory chapter extends to dense and discrete linear orders, and how the Until/Since-primitive language relates to derived Next/Previous operators and to a fresh-atom conservative-extension technique used internally by the metalogic.

Dependencies: Chapters 1–3 (syntax, semantics, proof theory).

5.1 The `FrameClass` Partial Order

The `FrameClass` classification that parameterizes `DerivationTree` (Section 2.1) is a three-element inductive type with an explicit partial order, defined in `ProofSystem/Axioms.lean`:

```
> Bimodal.ProofSystem.FrameClass.

inductive FrameClass where
  | Base
```

```
  | Dense
  | Discrete
```

`Base` is the bottom element (`FrameClass.base_le`); `Dense` and `Discrete` are incomparable extensions of `Base` – density and discreteness are jointly inconsistent frame properties, so no derivation may combine both. Every axiom constructor carries a minimum frame class via `Axiom.minFrameClass` (`ProofSystem/Axioms.lean:456-462`), and `DerivationTree`'s axiom rule enforces `ax.minFrameClass ≤ fc`:

Frame class	Axioms assigned
`Base`	37 axioms (layers 1–5): all base propositional, modal, BX temporal, interaction, and uniformity axioms
`Dense`	2 axioms: `density`, `dense_indicator`
`Discrete`	3 axioms: `prior_UZ`, `prior_SZ`, `z1`

Table 16: `Axiom.minFrameClass` breakdown (`ProofSystem/Axioms.lean:407-422` doc comment; counts regenerated by `scripts/typst-status-counts.sh`).

5.2 Semantic Frame-Condition Typeclasses

The syntactic `FrameClass` inductive and its order live entirely inside `ProofSystem/Axioms.lean`, above. The frame-condition **semantics** for dense and discrete orders lives elsewhere: `FrameConditions/FrameClass.lean` defines a separate typeclass hierarchy over a generic duration type `D`, used by the validity/soundness bridge below:

- `LinearTemporalFrame D` – the base bundle typeclass (`[AddCommGroup D] [LinearOrder D] [IsOrderedAddMonoid D]`).
- `SerialFrame D` – adds `Nontrivial`, `NoMaxOrder`, `NoMinOrder` (guarantees BX1/BX1′ seriality).
- `DenseTemporalFrame D` – adds `DenselyOrdered D`.
- `DiscreteTemporalFrame D` – adds `SuccOrder D`, `PredOrder D`, `IsSuccArchimedean D`.

`FrameConditions/Validity.lean` defines frame-relative validity over this hierarchy (`valid_over`, `valid_linear`, `valid_dense_fc`, `valid_discrete_fc`, and the concrete abbreviation `valid_over_Int`). `FrameConditions/Soundness.lean` and `Compatibility.lean` bridge this semantic layer to the syntactic `FrameClass`/`Axiom.minFrameClass` classification above (`soundness_linear`, `soundness_dense`, `soundness_discrete`, and the `AxiomLinearCompatible`/`AxiomDenseCompatible`/`AxiomDiscreteCompatible` typeclasses with per-axiom instances) – this is the machinery underlying the frame-class soundness theorems already stated in Chapter 15. All four

`FrameConditions/` files are sorry-free.[6]

5.3 Duration Groups and the Three Classes

The frame classes are best understood through the duration groups that realize them. Every duration type must be a nontrivial linearly ordered abelian group with no greatest or least element (the `SerialFrame` bundle), which is what validates the seriality axioms BX1/BX1′: every time has a strictly later and a strictly earlier time. Within that envelope, the two extensions pick out incompatible order structures:

Duration group	**Class**	**Why**
$\mathbb{Z}$	`Discrete`	Every element has an immediate successor and predecessor, and iterated succession reaches every later element
$\mathbb{Q}$, $\mathbb{R}$	`Dense`	Strictly between any two elements lies a third
$\mathbb{Z} \times \mathbb{Z}$ (lexicographic)	`Base` only	Not dense (nothing lies between $(0,0)$ and $(0,1)$), yet succession from $(0,0)$ never reaches $(1,0)$

Table 17: Representative duration groups for each frame class.

The lexicographic product is the instructive case: it shows that `Base` is not merely the union of the two extensions. A base-class derivation is sound over **every** admissible duration group, including orders that are neither dense nor discrete, which is why the base axioms must avoid both the density schema and the Prior/Z1 layer.

Density and discreteness exclude each other. The object-language witness is the discreteness indicator $U(\top, \bot)$ ("there is an immediate successor": the guard interval to the witness is empty). On a discrete order every point has an immediate successor, so $U(\top, \bot)$ holds everywhere; on a dense order the open interval between any point and any strictly later point is nonempty, so the guard $\bot$ can never hold throughout it and $U(\top, \bot)$ fails everywhere. A derivation combining the DI axiom $\neg U(\top, \bot)$ with the Prior/Z1 layer would therefore be sound over no frame at all, and the partial order on `FrameClass` – with `Dense` and `Discrete` incomparable – makes such a combination unformulable rather than merely unsound.

Worked validity: the density schema. DN ($GG\phi \to G\phi$) is valid over dense orders by a two-step argument. Suppose $GG\phi$ holds at time

[6]The prose doc comments inside `Soundness.lean` and `Compatibility.lean` cite axiom counts from an earlier stage of the axiom system; the counts used in this book are regenerated directly from source by `scripts/typst-status-counts.sh`.

x and let $y > x$ be arbitrary; density supplies z with $x < z < y$; from $GG\phi$ at x we get $G\phi$ at z, and since $y > z$ this gives ϕ at y. Over $\mathbb{Z}$ the argument breaks exactly at the appeal to density – there is no z strictly between x and its immediate successor – and indeed DN fails there: take ϕ true everywhere except at $x + 1$. Then $G\phi$ fails at x; but for every $y > x$, any $w > y$ satisfies $w \geq x + 2$, so $G\phi$ holds at every $y > x$ and $GG\phi$ holds at x.

Worked validity: the discrete layer. Prior-UZ ($F\phi \to U(\phi, \neg\phi)$) says that if ϕ holds somewhere in the future, it holds at a **nearest** future point – between now and that first witness, ϕ everywhere fails. Over $\mathbb{Z}$ this is the least-number principle applied to the set of future ϕ-times; over $\mathbb{Q}$ it fails, since $\{y : y > x, \phi \text{ at } y\}$ may have no least element (take ϕ to hold exactly on the open interval $(x, x + 1)$). Z1 ($G(G\phi \to \phi) \to (FG\phi \to G\phi)$) is the backward-induction principle of successor-Archimedean orders [6]: if ϕ is inductive along the strict future order and holds from some future point onward, discreteness lets the property propagate backward step by step to every future time. Both principles are axioms of frame class `Discrete` and are unavailable at `Base`, matching their failure over dense groups.

Monotonicity. Because `Base` sits below both extensions, every base-class theorem is automatically a theorem of both extended systems: `DerivationTree.lift` coerces derivations upward along the `FrameClass` order, and the soundness theorems for the three classes (`soundness_linear`, `soundness_dense`, `soundness_discrete`) confirm semantically that nothing is lost in the coercion.

5.4 Paper Correspondence: DF, DN, and CO[7]

The paper's Discreteness and Density frame properties correspond to the `DiscreteTemporalFrame`/`DenseTemporalFrame` typeclasses above, and are formalized via the `Discrete`/`Dense` `FrameClass` axiom layers. The paper's Completeness property (giving the $\mathbf{TM}_c$ and $\mathbf{TM}_{dc}$ extensions over Dedekind-complete flows of time) is a paper-side result (`possible_worlds.tex` `app:complete`): the formalization has no `Complete`-style typeclass and no matching `FrameClass` layer. This asymmetry (Discreteness/Density formalized, Completeness not) matches the three-element `FrameClass` type having no fourth constructor.

[7]The paper's Discreteness (DF), Density (DN), and Completeness (CO) frame properties, `possible_worlds.tex` §3.3 (:1162-1256) and appendices `app:discrete`/ `app:dense`/`app:complete`.

5.5 Next and Previous as Derived Operators

The paper's **Next** and **Previous** operators are derived, not primitive, in the Lean formalization. `Syntax/Formula.lean` defines them with the formula argument **first** and $\bot$ as the guard (the "Burgess convention" [2]: event first, guard second):

```
> Bimodal.Syntax.Formula.next.
```

```
def next (φ : Formula) : Formula := Formula.untl φ Formula.bot
-- X(φ) := φ U ⊥
def prev (φ : Formula) : Formula := Formula.snce φ Formula.bot
-- Y(φ) := φ S ⊥
```

not `⊥ U φ` as an event-second convention would give. $X(\phi)$ at t means ϕ holds at the immediate successor of t (event $= \phi$ at the immediate successor, guard $= \bot$ vacuously true at no strictly-intermediate time); dually for `prev`. The unfold lemmas confirming this characterization are `next_unfold` (`Automation/Normalization.lean:70`) and `prev_unfold` (`Automation/Normalization.lean:73`), both `@[simp]`-tagged and proven by `rfl`. The paper's own Next/Previous theorem is a paper-side result with no Lean counterpart.

5.6 A Fresh-Atom Conservative-Extension Technique

`Metalogic/ConservativeExtension/` is sorry-free, and its main theorem is a Goldblatt/BdRV-style fresh-atom naming lemma: extend the atom set with one fresh atom (ExtAtom $:=$ Atom $\oplus$ Unit, `ExtFormula.lean`), and any derivation over the extended language projects back down to a derivation over the original language, **provided** the fresh atom's own status is only used as a marker:

> **Theorem 5.1** **Conservative Lifting**
>
> For any frame class `fc`, context `L`, and formula `φ`, if the extended system derives `embedFormula φ` from `L.map embedFormula`, then the base system derives `φ` from `L`.[8]

This lemma is distinct from the paper's $\mathcal{L}$-vs-$\mathcal{L}^+$ (H/G-primitive vs. Until/Since-extended) conservative-extension theorem: no Lean module

[8] `lift_derivation_qfree` in `Metalogic/ConservativeExtension/Lifting.lean:683-695`. The proof collects the extended derivation's `inl`-atoms, picks a genuinely fresh atom outside that finite set (`exists_fresh_atom`), and projects the derivation back via `liftDerivationWith`. The module's own doc comment states this is "the key result enabling the irreflexivity proof" – i.e. it is infrastructure for Section 15.3's fresh-atom argument that irreflexivity is not modally definable, not a direct formalization of the paper's base-language-vs-Until/Since-extended-language split.

implements a base-language-vs-Until/Since-extended-language lifting theorem, so the paper's conservativity result for $\mathcal{L} \subseteq \mathcal{L}^+$ is a paper-side result (see also the Language Basis note of Chapter 15).

6 Metalogic

The metalogic for the bimodal logic **TM** relates the BX proof system of the previous chapter to the task semantics. Soundness is fully proven for all three frame classes. Completeness is approached through a canonical-model construction developed end-to-end for each frame class; the argument's open steps lie in the chronicle construction for the dense case and in the discrete transfer pipeline, and the completeness of **TM** remains an open problem. The chapter closes with the tableau-based decision procedure and the live module structure of `Metalogic/`.

6.1 Soundness

Soundness establishes that only logical consequences are derivable. It is proven separately for each frame class, matching the `FrameClass` parameter on derivations.

Theorem 6.1 **Soundness**

If $\Gamma \vdash \phi$ then $\Gamma \vDash \phi$.[9]

Theorem 6.2 **Frame-Class Soundness**

Derivability at frame class `Dense` (respectively `Discrete`) implies validity over densely ordered (respectively discrete) task frames.[10]

The proof proceeds by induction on the derivation structure:

- **Axioms**: Each of the 42 axiom constructors is proven valid on the frames of its minimum frame class (base axioms on all linear orders, `density`/`dense_indicator` on dense orders, `prior_UZ`/`prior_SZ`/`z1` on discrete orders)
- **Assumptions**: Assumed formulas are true by hypothesis
- **Modus ponens**: Validity preserved under implication elimination
- **Necessitation**: Valid formulas become necessarily valid
- **Temporal necessitation**: Valid formulas become always-future valid
- **Temporal duality**: Past-future swap preserves validity
- **Weakening**: Adding premises preserves semantic consequence

[9] Proven sorry-free as `soundness` in `Metalogic/Soundness.lean`.

[10] Proven sorry-free as `soundness_dense` in `Metalogic/DenseSoundness.lean` and `soundness_discrete` in `Metalogic/DiscreteSoundness.lean`.

The axiom validity lemmas live in `Metalogic/SoundnessLemmas/` (with `Core.lean`, `DenseValidity.lean`, and `FrameClassVariants.lean`), and the frame-condition semantics for the Base/Dense/Discrete classes is developed in the top-level `FrameConditions/` directory. The modal-temporal interaction axiom MF uses time-shift invariance (via `time_shift` on world histories) to relate truth at different times.

6.2 Core Infrastructure

The completeness proof requires three foundational components: the deduction theorem, maximal consistent sets, and Lindenbaum's lemma.

6.2.1 Deduction Theorem

Theorem 6.3 **Deduction Theorem**

If $A :: \Gamma \vdash B$ then $\Gamma \vdash A \to B$.[11]

The proof uses well-founded induction on derivation height, handling each of the following rules:

- **Axiom**: Use S axiom to weaken
- **Assumption**: Identity if same, S axiom if different
- **Modus ponens**: Use K axiom distribution
- **Weakening**: Case analysis on assumption membership
- **Modal/temporal rules**: Do not apply (require empty context)

6.2.2 Consistency

Definition 6.4 **Consistent**

A context Γ is **consistent** if $\Gamma \nvdash \bot$.

Definition 6.5 **Maximal Consistent**

A set of formulas S is **maximal consistent** if it is consistent and for all $\phi \notin S$, the set $S \cup \{\phi\}$ is inconsistent.[12]

Maximal consistent sets (MCS) are negation-complete: for every formula ϕ, exactly one of ϕ or $\neg\phi$ is a member. This property is essential for canonical constructions, as it ensures that every formula has a definite truth value in each MCS.

[11] Proven in `Metalogic/Core/`; see `deduction_theorem`.

[12] Formalized at the set level as `SetMaximalConsistent`, parameterized by frame class.

6.2.3 Lindenbaum's Lemma

Lemma 6.6 — **Lindenbaum**

Every consistent set of formulas extends to a maximal consistent set.[13]

The proof applies Zorn's lemma to the partially ordered collection of consistent supersets of the given set. The key step is showing that the union of any chain of consistent sets is itself consistent: any derivation of $\bot$ uses only finitely many premises, which would be contained in some member of the chain, contradicting that member's consistency.

6.3 Completeness

The completeness theorem is stated for each frame class, with the base-class theorem as the primary entry point.

Theorem 6.7 — **Completeness (Base)**

If $\vDash \phi$, then there is a derivation of ϕ at frame class `Base`.[14]

Theorem 6.8 — **Completeness (Dense, Discrete)**

Validity over densely ordered frames implies derivability at frame class `Dense`; validity over discrete frames implies derivability at frame class `Discrete`.[15]

6.3.1 Proof Architecture

The proof is by contraposition. If ϕ is not derivable, then $\{\neg\phi\}$ is consistent, and Lindenbaum's lemma extends it to an MCS M containing $\neg\phi$. A countermodel for ϕ is then built from M by a three-way case split on the discreteness indicator $U(\top, \bot)$ ("there is an immediate successor"):

1. **Dense case** ($\Box\neg U(\top, \bot) \in M$): a countermodel is constructed on the rational timeline $\mathbb{Q}$ via the Burgess-style **chronicle construction** [2].[16]
2. **Discrete case** ($\Box U(\top, \bot) \in M$): a countermodel is constructed on the integer timeline $\mathbb{Z}$ via the Reynolds/Doets pipeline [5], [6].[17]

---

[13] Proven as `set_lindenbaum` in `Metalogic/Completeness.lean`.

[14] Stated as `completeness` in `Metalogic/BXCanonical/Completeness.lean`: `valid φ → Nonempty (DerivationTree FrameClass.Base [] φ)`. The open steps of the proof are described at the end of this section.

[15] Stated as `completeness_dense` and `completeness_discrete` in `Metalogic/BXCanonical/Completeness.lean`.

[16] `Metalogic/BXCanonical/Chronicle/`, entry point `countermodel_dense` in `ChronicleToCountermodelBasic.lean`, drawing on the D-parametric truth lemma in `Metalogic/Algebraic/`.

[17] `Metalogic/WeakCanonical/`, transfer step in `WeakCanonical/Transfer.lean`.

3. **Mixed case**: eliminated outright — an MCS cannot be undecided about discreteness.[18]

The construction rests on shared infrastructure:

- **Bundled families of MCSs** (`Metalogic/Bundle/`): time-indexed families of maximal consistent sets with G/H coherence conditions, used by all completeness paths.
- **Algebraic parametric completeness** (`Metalogic/Algebraic/`): a truth lemma parametric in the duration type D, which turns a coherent MCS family into a task model.
- **Chronicles** (`Metalogic/BXCanonical/Chronicle/`): the Burgess [2] dense-order construction, filling in witnesses for Until/Since eventualities over $\mathbb{Q}$.
- **Filtration and quasimodels** (`Metalogic/BXCanonical/Filtration/`, `Quasimodel/`): finitary approximations used in the canonical chain construction.

6.3.2 Open Steps in the Completeness Argument

The completeness of **TM** with respect to its frame classes is an open problem. The open steps are concentrated in the chronicle construction (coherence of the constructed MCS family) and in the discrete-case truth lemma and transfer (`WeakCanonical/TruthLemma.lean`, `Transfer.lean`, and the Kamp-style expressiveness modules [7]). The discrete path runs through a Kamp-theorem-based expressive-completeness argument, developed in `WeakCanonical/`, which likewise remains open.

6.4 Decidability

The decision procedure for **TM** is tableau-based, with `decide_sound` (soundness) proven sorry-free and the finite-model-property completeness result (`fmp_completeness`) also sorry-free as a finite-filtration statement whose semantic-validity bridge is separately open. The full operational account – entry points, fuel semantics, certificate/countermodel extraction, the finite-model-property statement, and the tableau complexity table – is given in Chapter 7.

6.5 Module Structure

The live metalogic code is organized as follows (`Theories/Bimodal/Metalogic/`):

[18] `mcs_mixed_case_absurd` in `Metalogic/BXCanonical/Chronicle/MCSMixedCase.lean`.

Module	Contents
`Core/`	MCS theory, provability interface, deduction theorem
`Bundle/`	Time-indexed MCS families (BFMCS) with coherence conditions
`Algebraic/`	D-parametric algebraic completeness and truth lemma
`BXCanonical/`	Completeness theorem; `Chronicle/` (dense case), `Filtration/`, `Quasimodel/`
`WeakCanonical/`	Reynolds/Doets discrete completeness path; `Separation/`; Kamp-style expressiveness modules
`ConservativeExtension/`	Conservative extension results
`Decidability/`	Tableau decision procedure; `FMP/` finite model property
`Relational/`	Relational semantics bridges
`SoundnessLemmas/`	Per-axiom validity lemmas, dense/discrete variants
`Soundness.lean`	Soundness theorem (sorry-free)
`DenseSoundness.lean`, `DiscreteSoundness.lean`	Frame-class soundness (sorry-free)
`Completeness.lean`	MCS properties and Lindenbaum lemma
`Decidability.lean`	Decidability interface

Table 18: Live `Metalogic/` module structure.

6.6 Implementation Status

Soundness in all three frame-class variants, the deduction theorem, the MCS/Lindenbaum infrastructure, the perpetuity principles P1–P6, and the entire `Syntax/`, `Semantics/`, `ProofSystem/`, and `Theorems/` trees are fully proven in Lean under `Theories/Bimodal/`. The canonical-model construction toward completeness is developed end-to-end for each frame class, with the open steps localized as described above; completeness itself remains an open problem. The decision procedure's soundness (`decide_sound`) and the finite-filtration FMP statement

(fmp_completeness) are likewise proven, with the semantic-validity bridge treated in Chapter 7.

6.6.1 Semantic Convention

The completeness architecture is built for the **strict (irreflexive) temporal semantics** of Section 3.4: G and H quantify over strictly future and strictly past times, so the temporal T-axioms $G\phi \to \phi$ and $H\phi \to \phi$ are **not** valid, and seriality is supplied axiomatically by BX1/ BX1′.[19]

7 Decidability in Practice

The operational decision procedure – entry points, fuel semantics, certificates, and countermodels – presented as a usable artifact, together with a precise account of what is and is not proven about it.

Dependencies: Chapters 1–3; Chapter 15 for the strict-semantics convention.

7.1 The Finite Model Property

The completeness side of the decision procedure rests on a finite model property, developed in Metalogic/Decidability/FMP/. The entire Metalogic/Decidability/ tree, including all seven FMP/ files, is sorry-free, and the central theorem is fmp_completeness (Decidability/Correctness.lean:123):

Theorem 7.1 **FMP-Based Completeness**

If φ is a member of every closure maximal-consistent-set bundle for φ, then φ is derivable at frame class Base.[20]

The antecedent quantifies over FMP.ClosureMCSBundle φ – a **finite, syntactic** filtration structure (Filtration.lean:114: a carrier set plus an is_mcs witness) – not directly over semantic validity $\vDash \varphi$ across every task model of every duration type. The finite-filtration completeness argument is therefore complete as it stands, while the bridge connecting "true in every closure MCS bundle" to semantic validity in the task-

[19] Semantics/Truth.lean and the module docstring of Metalogic/Metalogic.lean document this convention.

[20] fmp_completeness (φ : Formula) : (∀ (S : FMP.ClosureMCSBundle φ), φ ∈ S.carrier) → Nonempty (DerivationTree FrameClass.Base [] φ), Metalogic/Decidability/Correctness.lean:123-126, proved (sorry-free) by FMP.fmp_contrapositive (FMP/FMP.lean:206), itself built from FMP.mcs_finite_model_property (FMP/FMP.lean:193) over the finite quotient type FilteredWorld (FMP/Filtration.lean:152, finiteness witnessed sorry-free by FilteredWorld.finite, FMP/FiniteModel.lean:131).

frame sense of Section 3.4 is an open problem. The distinction matters when citing the result: `fmp_completeness` is a theorem about the filtration structure, not a theorem about task-frame validity.

7.1.1 The Filtration Construction

The finiteness argument follows the classical filtration pattern, specialized to closure MCSs. Everything is relativized to the input formula φ: the **subformula closure** of φ collects its subformulas and their negations, a finite set; a **closure MCS** is a maximal consistent set restricted to that closure (`ClosureMCS`, with the bundled form `ClosureMCSBundle` pairing a carrier set with its maximality witness, `Filtration.lean:114`). Two closure MCSs are **filtration-equivalent** when they agree on every closure formula, and the quotient of bundles by this equivalence is the finite world type:

```
> Bimodal.Metalogic.Decidability.FMP.FilteredWorld.
```

```
def FilteredWorld (phi : Formula) : Type :=
  Quotient (ClosureMCSSetoid phi)
```

The number of equivalence classes is bounded by $2^{|\,\mathrm{closure}(\varphi)|}$, and finiteness is witnessed constructively by `FilteredWorld.finite` (`FMP/FiniteModel.lean:131`). From there, `FMP.mcs_finite_model_property` (`FMP/FMP.lean:193`) shows that a formula outside some closure MCS fails in the induced finite structure, and `FMP.fmp_contrapositive` (`FMP/FMP.lean:206`) turns this into the completeness form quoted above: membership in **every** closure MCS bundle forces derivability. Truth preservation across the quotient is handled in `FMP/TruthPreservation.lean`, and the dense and discrete refinements live in `FMP/DenseFMP.lean` and `FMP/DiscreteFMP.lean`. The construction is thus a complete, self-contained finite combinatorics of the proof system; what it does not supply – the open bridge noted above – is the semantic step identifying "true in every closure MCS bundle" with task-frame validity.

7.2 The Decision Procedure

`decide` is the top-level entry point:

```
> Bimodal.Metalogic.Decidability.decide.
```

```
def decide (φ : Formula) (searchDepth : Nat := 10)
(tableauFuel : Nat := 1000)
    (fc : FrameClass := .Base) : DecisionResult φ
```

(`Decidability/DecisionProcedure.lean:122`). It first tries direct axiom and compositional proof shortcuts, then falls back to bounded proof search (Chapter 12), then to a tableau over $F(\varphi)$; `DecisionResult` (`Decidability/DecisionProcedure.lean:58-64`) is one of `valid` (carries a `DerivationTree`), `invalid` (carries a

SimpleCountermodel), or timeout – the tableauFuel parameter (default 1000 steps) is the source of the timeout branch, guaranteeing termination without guaranteeing an answer. Convenience wrappers isValid (Decidability/DecisionProcedure.lean:163) and isSatisfiable (Decidability/DecisionProcedure.lean:169) reduce to booleans. getProof? (Decidability/DecisionProcedure.lean:87) and getCountermodel? (Decidability/DecisionProcedure.lean:92) extract the payload when present.

7.3 Certificates and Countermodels

Every valid result carries a genuine DerivationTree proof term, checkable by Lean's kernel independently of the decision procedure that produced it. TraceCertificate.lean additionally packages proof search traces: ProofCertificate (Decidability/TraceCertificate.lean:108, with an empty constructor at Decidability/TraceCertificate.lean:137) records the rule sequence, and CertOutcome (Decidability/TraceCertificate.lean:89: validProof/countermodel/timeout/blocked) classifies the outcome for downstream export (feeding Part II's dataset pipeline). Every invalid result carries a SimpleCountermodel (Decidability/CountermodelExtraction.lean:64: trueAtoms/falseAtoms/formula), extracted from the open saturated tableau branch by extractCountermodelSimple (Decidability/CountermodelExtraction.lean:137, called directly by decide) or, for a richer variant retaining the full saturated branch, extractSemanticCountermodel (Decidability/CountermodelExtraction.lean:305) producing a SemanticCountermodel (Decidability/CountermodelExtraction.lean:170).

7.4 Correctness Properties

- decide_sound (Decidability/Correctness.lean:50): if decide returns valid with proof π, then $\vDash \varphi$. Proven, sorry-free – this is the load-bearing correctness guarantee: every "valid" answer is backed by a kernel-checked proof term, so it is trustworthy independent of the decision procedure's own correctness.
- validity_decidable (Decidability/Correctness.lean:72): stated as (⊨ φ) ∨ ¬(⊨ φ). Its proof is literally Classical.em (⊨ φ) – this is **not** a constructive decidability result and should never be cited as one; it is a vacuous classical tautology, included in the file for documentation purposes only.
- fmp_completeness: sorry-free as a finite-filtration statement; its semantic-validity bridge is open (Section 7.1).

- The tableau's own completeness (every semantically valid formula's tableau eventually closes, within fuel) is not separately proven in this module; `decide_sound` covers only the soundness direction.

7.5 A Worked Tableau Run

The tableau operates on **signed formulas**: each node of a branch is a formula tagged with a sign (T for "true here", F for "false here") and a label recording the world and time of evaluation (`SignedFormula`, `Metalogic/Decidability/SignedFormula.lean`). Expansion rules (`TableauRule`, `Metalogic/Decidability/Tableau.lean:67`) come in two shapes: **linear** rules add formulas to the current branch, and **branching** rules split it. The modal and temporal rules follow the S5 and strict-linear-order semantics directly: a true $\Box$-formula propagates to every world label on the branch (universal, persistent), a false $\Box$-formula introduces a fresh witness world, a false G-formula introduces a fresh strictly-future time, and a true $\Box$-formula additionally yields G and H versions at the same label – the tableau image of the interaction principles $\Box\phi \to G\phi$ and $\Box\phi \to H\phi$. A branch **closes** when it contains both $T(\phi)$ and $F(\phi)$ at the same label (or $T(\bot)$); the tableau proves validity when every branch closes.

Two miniature runs illustrate both outcomes.

A closing tableau. To decide the MT instance $\Box p \to p$, the procedure refutes its negation, seeding the branch with the signed formulas $T(\Box p)$ and $F(p)$ at the initial label (w_0, t_0). The `boxPos` rule propagates $T(p)$ to every known world at t_0 – in particular to w_0 itself – and the branch now contains $T(p)$ and $F(p)$ at the same label: closed. Since the (single) branch closes, the formula is valid, and proof extraction (`Metalogic/Decidability/ProofExtraction.lean`) returns a `DerivationTree` certificate.

An open tableau. To decide $p \to Gp$, the branch is seeded with $T(p)$ and $F(Gp)$ at (w_0, t_0). The `allFutureNeg` rule introduces a fresh time $t_1 > t_0$ carrying $F(p)$. No rule ever forces $T(p)$ at t_1 – the true atom at t_0 is not a universal – so the branch saturates open. The open saturated branch is precisely a countermodel description: p true at (w_0, t_0), false at (w_0, t_1), which `extractCountermodelSimple` packages as a `SimpleCountermodel` refuting the formula.

7.6 Complexity

The operational costs of the procedure are driven by two parameters. The number of distinct signed formulas on a branch is bounded by the closure of the input formula (its subformulas and their negations) times the number of labels the run introduces, so each branch is polynomial in

the input size for a fixed label budget; the number of **branches**, however, can grow exponentially in the number of branching-rule applications, giving worst-case exponential time overall. Fresh-witness rules (`boxNeg`, `allFutureNeg`, and their duals) are the source of new labels, and universal rules must revisit each new label, which is why the implementation tracks universals as **persistent** and existentials as **consumable**. The `tableauFuel` parameter bounds the total number of expansion steps: fuel-bounded termination trades completeness-within-fuel for a hard guarantee against non-termination, and an exhausted budget surfaces as the `timeout` outcome rather than as a wrong answer.

7.7 Closing Status Table

Fragment / Property	Status
Decision soundness (`decide_sound`)	Proven, sorry-free
Finite-model-property completeness (`fmp_completeness`)	Proven, sorry-free (finite-filtration statement)
Semantic-validity bridge for FMP	Open problem
Constructive validity decidability	Open problem (`validity_decidable` is vacuous)
Fully automated decidable fragment	Open problem

Table 19: Decidability status summary.

8 Theorems

8.1 Perpetuity Principles

The perpetuity principles establish deep connections between modal necessity ($\Box$) and temporal operators ($\triangle$, $\triangledown$); they are the touchstone principles P1–P6 of [1], proven here in Lean (`Theorems/Perpetuity/`).

Theorem 8.1 **P1: Necessity Implies Always**

$\vdash \Box\phi \to \triangle\,\phi$

Theorem 8.2 **P2: Sometimes Implies Possible**

$\vdash \triangledown\,\phi \to \Diamond\phi$

Theorem 8.3 **P3: Necessity of Perpetuity**

$\vdash \Box\phi \to \Box \triangle \phi$

Theorem 8.4 **P4: Possibility of Occurrence**

$\vdash \Diamond \triangledown \phi \to \Diamond\phi$

Theorem 8.5 **P5: Persistent Possibility**

$\vdash \Diamond \triangledown \phi \to \triangle \Diamond\phi$

Theorem 8.6 **P6: Occurrent Necessity is Perpetual**

$\vdash \triangledown \Box\phi \to \Box \triangle \phi$

All six perpetuity principles are fully proven (sorry-free) in the Lean implementation, in `Theorems/Perpetuity/` (`Principles.lean`, with P6 infrastructure in `Bridge.lean`).

Principle	Lean Theorem	Key Lemmas
P1	`perpetuity_1`	MF, TF, MT
P2	`perpetuity_2`	Contraposition of P1
P3	`perpetuity_3`	P1, box_mono
P4	`perpetuity_4`	Contraposition
P5	`perpetuity_5`	modal_5, temporal K
P6	`perpetuity_6`	P5, bridge lemmas

8.2 Modal S5 Theorems

Theorem 8.7 **T-Box-to-Diamond**

$\vdash \Box\phi \to \Diamond\phi$

Theorem 8.8 **Box Distributes Over Disjunction**

$\vdash (\Box\phi \vee \Box\psi) \to \Box(\phi \vee \psi)$

Theorem 8.9 **Box Preserves Contraposition**

$\vdash \Box(\phi \to \psi) \to \Box(\neg\psi \to \neg\phi)$

Theorem 8.10 **K Distribution for Diamond**

$\vdash \Box(\phi \to \psi) \to (\Diamond\phi \to \Diamond\psi)$

Theorem 8.11 **S5 Collapse**

$\vdash \Diamond\Box\phi \leftrightarrow \Box\phi$

Theorem 8.12 **Box-Conjunction**

$\vdash \Box(\phi \wedge \psi) \leftrightarrow (\Box\phi \wedge \Box\psi)$

Theorem 8.13 **Diamond-Disjunction**

$\vdash \Diamond(\phi \vee \psi) \leftrightarrow (\Diamond\phi \vee \Diamond\psi)$

Theorem 8.14 **S5 Diamond-Box to Truth**

$\vdash \Diamond\Box\phi \rightarrow \phi$

Theorem 8.15 **T-Box Consistency**

$\vdash \Box(\phi \wedge \neg\phi) \rightarrow \bot$

8.3 Modal S4 Properties

The following S4 properties are derived from the **TM** axiom system.

Theorem 8.16 **Modal 5**

$\vdash \Diamond\phi \rightarrow \Box\Diamond\phi$

Theorem 8.17 **Diamond 4**

$\vdash \Diamond\Diamond\phi \rightarrow \Diamond\phi$

Theorem 8.18 **Box Monotonicity**

If $\vdash \phi \rightarrow \psi$ then $\vdash \Box\phi \rightarrow \Box\psi$.

Theorem 8.19 **Diamond Monotonicity**

If $\vdash \phi \rightarrow \psi$ then $\vdash \Diamond\phi \rightarrow \Diamond\psi$.

8.4 Propositional Theorems

Theorem 8.20 **Identity**

$\vdash \phi \rightarrow \phi$

Theorem 8.21 **Double Negation Introduction**

$\vdash \phi \to \neg\neg\phi$

Theorem 8.22 **Double Negation Elimination**

$\vdash \neg\neg\phi \to \phi$

Theorem 8.23 **Contraposition**

If $\vdash \phi \to \psi$ then $\vdash \neg\psi \to \neg\phi$.

Theorem 8.24 **De Morgan Disjunction**

$\vdash \neg(\phi \vee \psi) \leftrightarrow (\neg\phi \wedge \neg\psi)$

Theorem 8.25 **De Morgan Conjunction**

$\vdash \neg(\phi \wedge \psi) \leftrightarrow (\neg\phi \vee \neg\psi)$

8.5 Combinator Infrastructure

The combinator infrastructure provides Hilbert-style proof tools.

Theorem 8.26 **B Combinator**

$\vdash (B \to C) \to ((A \to B) \to (A \to C))$

Theorem 8.27 **Implication Transitivity**

If $\vdash A \to B$ and $\vdash B \to C$ then $\vdash A \to C$.

Theorem 8.28 **Pairing**

$\vdash A \to (B \to (A \wedge B))$

Theorem 8.29 **Classical Merge**

$\vdash (P \to Q) \to ((\neg P \to Q) \to Q)$

8.6 Generalized Necessitation

Theorem 8.30 **Generalized Modal Necessitation**

If $\Gamma \vdash \phi$ then $\Box\Gamma \vdash \Box\phi$ where $\Box\Gamma = [\Box\psi \mid \psi \in \Gamma]$.

Theorem 8.31 **Generalized Temporal Necessitation**

If $\Gamma \vdash \phi$ then $G\Gamma \vdash G\phi$ where $G\Gamma = [G\psi \mid \psi \in \Gamma]$.

8.7 Module Organization

The `Theorems/` directory is organized as follows:

Module	Contents
`Perpetuity/`	P1–P6 principles (`Principles.lean`, `Bridge.lean`, `Helpers.lean`); re-exported by `Perpetuity.lean`
`Propositional/`	Classical propositional theorems (`Core.lean`, `Connectives.lean`, `Reasoning.lean`)
`ModalS5.lean`	S5 characteristic theorems
`ModalS4.lean`	S4 properties (modal_5, diamond_4)
`Combinators.lean`	B, I, S combinators, imp_trans, `temp_future_derived` (TF)
`TemporalDerived.lean`	Derived temporal axioms: `temp_k_dist_derived` (TK), `temp_4_derived` (T4)
`ContextualProofs.lean`	Derivations at non-empty contexts
`GeneralizedNecessitation.lean`	Context-level necessitation

The entire `Theorems/` tree is sorry-free.

9 From LTL to TM

*Where **TM** sits relative to the temporal logics of the model-checking literature: what is genuinely shared with LTL, what is different by design, and why the difference is a matter of semantic architecture rather than a bolted-on modality.*

Dependencies: Chapters 1–3 (syntax, semantics, proof theory); Section 15.3 for the strict-semantics convention.

9.1 The Thesis

Readers arriving from computer science will recognize much of **TM**'s operator vocabulary from linear temporal logic [8], [9], [10], and the recognition is legitimate – but the identification is not. Stated precisely:

TM is Until/Since temporal logic over linear orders – durations form a totally ordered abelian group, and the operators quantify strictly – fused with an S5 metaphysical modality through the interaction axiom MF and the uniformity axioms, interpreted over task frames in which possible worlds are task-constrained functions from convex sets of durations to world states. The modality is not a second primitive Kripke dimension laid alongside the temporal one: $\Box$ quantifies over the *constructed* history space $H_{\mathcal{F}}$, and it is the time-shift invariance of that construction that validates the perpetuity principles connecting the two operator families (see the Time-Shift section of the semantics chapter and the perpetuity theorems P1–P6 of the theorems chapter; they are not restated here). Every contrast drawn in this chapter unpacks some part of that sentence.

9.2 Traces versus Task Frames

LTL is interpreted over traces: ω-sequences of states (or of atomic-proposition labelings), with a distinguished initial point, a primitive next-step operator, and a non-strict Until [8], [9], [10]. **TM** is interpreted over the task frames of the semantics chapter, and the differences are structural rather than notational:

- **Time domain.** A trace is indexed by $\mathbb{N}$; a **TM** history $\tau : X \to W$ has a convex domain X in an arbitrary totally ordered abelian group of durations – discrete, dense, or neither – and may be bi-infinite or partial.
- **Initial point.** A trace begins somewhere, and LTL validity is anchored at that initial position; a task-frame history has no distinguished start, and **TM** validity is floating – quantified over all frames, histories, *and* times.
- **The underlying system.** A task frame is itself a labeled transition system in which the transition labels are durations carrying group structure [1], [9], so the model-checking reading of "system plus runs" transfers directly: a history is a labeled run of the system. What LTL calls the set of traces of a system, **TM** constructs as the history space $H_{\mathcal{F}}$ – and then quantifies over it with $\Box$ in the object language.

The semantics chapter (Section 3.4) gives the full truth conditions; nothing in this comparison modifies them.

9.3 Operator Conventions

Against the shared Until vocabulary, four conventions differ, each adjudicated elsewhere in this book and only collected here:

Convention	LTL	TM
Until reading	Non-strict (present included)	Strict/irreflexive (Section 15.3)
Next step	Primitive operator	Derived: `next`/`prev` as $\phi\, U\, \bot$ / $\phi\, S\, \bot$, Burgess convention, unfold lemmas `next_unfold`/ `prev_unfold` (frame-classes chapter)
Past operators	Absent (future-only)	Present: Since is primitive alongside Until
Validity	Anchored at the initial position	Floating over all frames, histories, and times

Table 22: Operator-convention deltas between LTL and **TM**.

The future-only convention of LTL is principled, not an oversight: over discrete $\mathbb{N}$-like flows the past-free fragment is already expressively adequate – the Kamp-theorem discussion of Chapter 10 explains why, and why the reduction is unavailable over **TM**'s general duration groups, where Since does independent work.

9.4 Translating LTL into TM

The convention deltas above compose into a systematic translation. An LTL trace – an ω-sequence of states – embeds into the task-frame setting as a history over the duration group $\mathbb{Z}$ with convex domain $\{x : x \geq 0\}$, realized in the frame whose task relation is trivial: the identity at duration 0 and the total relation at every other duration (this relation satisfies the nullity, reflection, and compositionality constraints outright). Position i of the trace becomes time i of the history, and atoms are false at times outside the domain, matching the semantics chapter's convention. On formulas, each non-strict LTL operator has a strict **TM** rendering:

LTL operator	TM rendering	Comment
$X\phi$	$U(\phi, \bot)$	`next`, exact over discrete frames
$\psi\, U^{\mathrm{ns}} \phi$	$\phi \vee (\psi \wedge U(\phi, \psi))$	Non-strict Until: witness now, or strictly later with the guard from now
$F^{\mathrm{ns}}\phi$	$\phi \vee F\phi$	Reflexive "eventually"
$G^{\mathrm{ns}}\phi$	$\phi \wedge G\phi$	Reflexive "henceforth"

Table 23: Non-strict LTL operators rendered over **TM**'s strict primitives.

The translation is truth-preserving position by position, and two worked examples show where the care pays off.

Next and self-duality. LTL validates $\neg X\phi \leftrightarrow X\neg\phi$: every trace position has exactly one successor, so "not: ϕ next" and "next: not ϕ" coincide. In **TM** the rendering $U(\phi, \bot)$ makes this equivalence a **frame-class** fact rather than a logical one. Over $\mathbb{Z}$, $U(\phi, \bot)$ holds at x exactly when ϕ holds at $x + 1$, and self-duality follows; over a dense order, no point has an immediate successor, so $U(\phi, \bot)$ is false for **every** ϕ – both $X\phi$ and $X\neg\phi$ fail while $\neg X\phi$ holds. The equivalence is thus valid over discrete frames and invalid over dense ones: what LTL builds into its models, **TM** exposes as a frame-class distinction.

Anchoring. LTL validity is truth at position 0 of every trace; **TM** validity is floating over all frames, histories, and times. The translation preserves truth pointwise, so it maps LTL **truth conditions** faithfully, but validity does not transfer in either direction without an anchoring argument: an LTL-valid formula's rendering must be checked at arbitrary evaluation points (not just a designated start), and over arbitrary duration groups (not just $\mathbb{N}$-like ones). The frame-class machinery is exactly the instrument for that check – renderings that depend on discreteness, like the Next self-duality above, hold over discrete frames and not in general.

9.5 Neither Fusion nor Product

The many-dimensional modal logic literature provides the right vocabulary for the combination itself [11]. Two stock recipes combine a temporal logic with S5: the *fusion*, which imposes no interaction between the operator families, and the *product*, which imposes full commutation interaction by evaluating at pairs of a time and a world. **TM** is neither. It is more than the fusion, because MF (and the derived TF) are theorems relating $\Box$ to the temporal operators; and it is not literally the product of a linear tense logic with S5, because its interaction principles are not *imposed* on a two-dimensional grid of evaluation points – they *hold* in virtue of the construction of the history space, whose time-shift invariance makes perpetuity come out valid [1]. In the general product landscape such principles must be stipulated frame condition by frame condition; here a single semantic decision – worlds are shift-invariant families of task-respecting functions – yields them wholesale. This distinction matters again at the decidability frontier, since products and fusions have sharply different computational profiles – the final chapter of this part surveys that landscape.

9.6 Linear, Branching, and Hyper

The paper positions **TM** inside the classical triangle of linear-time, branching-time, and hyper logics [1], and the coordinates are worth restating. Individual possible worlds are linearly ordered, as in LTL [8]; the set of possible worlds passing through a given world state branches, as in CTL [12]; and CTL$\star$ combines path quantifiers with temporal operators freely [13], in the debate initiated by Lamport [14] and pressed to its "final showdown" by Vardi [15]. The key difference from the branching-time family is that $\Box$ quantifies over *all* possible worlds in $H_{\mathcal{F}}$ at the evaluation time, rather than employing the state-relativized path quantifiers A and E of CTL; it is precisely this unrestricted quantification, combined with time-shift invariance, that validates perpetuity. HyperLTL provides the sharpest contrast: it extends LTL with quantifiers $\forall\pi$ and $\exists\pi$ that bind trace variables in the object language [16], and that binding power makes its satisfiability problem undecidable [17]. **TM**'s $\Box$ binds nothing – it is a modal operator over the constructed history space, not a first-order quantifier over traces – and the language stays on the modal side of that line. The cost profile of crossing it is the business of Chapter 11.

9.7 Branching-Time Neighbors

Two philosophical traditions posit branching outright. The STIT theory of Belnap, Perloff, and Xu [18] and Rumberg's transition semantics [19] – following Thomason's reconstruction of the Ockhamist tradition [20] – take the temporal order itself to be a partial order with incomparable moments branching toward the future. **TM** keeps the duration order total and locates indeterminism in the task relation instead: the relation $\underset{x}{\Longrightarrow}$ need not be functional, so the space of histories through a world state *unfolds into* a tree without a tree ever being posited. When the state space is finite and durations are the integers, the complete histories form a shift of finite type in the sense of symbolic dynamics [21] – the finite-discrete instance of the same construction.

9.8 The Conservativity Bridge

One last bookkeeping point lets this chapter compare **TM** with LTL operator-by-operator without changing systems mid-argument, and it has a paper half and a Lean half that must not be conflated. On the paper side, **TM** over the tense basis extends conservatively to $\mathbf{TM}^{+}$ over the Until/Since basis [1], and the book's Lean system works in the Until/Since basis natively – `untl` and `snce` are the primitive constructors, with $H/G/F/P$ derived (see the Language Basis note of

Chapter 15) – so the LTL-style operator vocabulary is the native one here, and the tense fragment is recovered inside it. On the Lean side, the finer statement of Section 5.6 applies: no Lean module formalizes the paper's base-language-versus-extended-language conservativity theorem; `Metalogic/ConservativeExtension/` proves the fresh-atom lifting lemma `lift_derivation_qfree`, infrastructure for the irreflexivity argument rather than the paper's theorem. The comparison in this chapter therefore rests on the paper's theorem as a paper-side result, and on the fact that the formalized system itself simply *is* an Until/Since system.

10 Vlach Operators and the BL* Tower

The store/recall operators that extend the bimodal language with cross-referencing power over both times and worlds, the resulting language BL*, and the hybrid-logic prior art against which both are positioned – including Kamp's expressive-completeness theorem, correctly scoped.

Dependencies: Chapters 1–3 (syntax, semantics, proof theory); Section 15.3 for the strict-semantics convention.

10.1 Why Cross-Reference?

The systems developed so far – **TM** over the tense basis and **TM**$^+$ over the Until/Since basis – are expressive enough for a wide range of temporal-modal reasoning, yet they lack the means by which to cross reference either times or worlds [1]. Natural language routinely performs exactly this kind of cross-referencing. Consider the sentence "Once, everyone now alive hadn't yet been born." Its truth conditions require evaluating "now alive" at the utterance time even though the phrase sits under a past-tense operator that has shifted the evaluation time; no nesting of P, F, S, and U reproduces this behavior, because every operator of **TM**$^+$ shifts the point of evaluation without remembering where evaluation began. Kamp introduced the rigid *now* operator to handle such sentences [7], and Vlach generalized the idea with a *then* operator that recalls a previously stored time [22]. The same phenomenon arises in the world dimension: hypothetical and modal discourse refers back to the world of evaluation from within the scope of a modal.

10.2 The Store and Recall Operators

Following [1], the point of evaluation is extended with a vector $\vec{v} = \langle v_1, v_2, ... \rangle$ of stored times and a vector $\vec{\mu} = \langle \mu_1, \mu_2, ... \rangle$ of stored worlds. Four indexed families of operators read and write these registers, for each $i \in \mathbb{N}$:

Definition 10.1 **Store and Recall**

Writing $\vec{v}[i \mapsto x]$ for the result of overwriting the i^{th} coordinate of $\vec{v}$ with x (and similarly for $\vec{\mu}$):

$$\mathcal{M}, \tau, x, \vec{v}, \vec{\mu} \vDash \overset{i}{\uparrow} \phi \iff \mathcal{M}, \tau, x, \vec{v}[i \mapsto x], \vec{\mu} \vDash \phi$$

$$\mathcal{M}, \tau, x, \vec{v}, \vec{\mu} \vDash \overset{i}{\downarrow} \phi \iff \mathcal{M}, \tau, v_i, \vec{v}, \vec{\mu} \vDash \phi$$

$$\mathcal{M}, \tau, x, \vec{v}, \vec{\mu} \vDash \overset{i}{\Uparrow} \phi \iff \mathcal{M}, \tau, x, \vec{v}, \vec{\mu}[i \mapsto \tau] \vDash \phi$$

$$\mathcal{M}, \tau, x, \vec{v}, \vec{\mu} \vDash \overset{i}{\Downarrow} \phi \iff \mathcal{M}, \mu_i, x, \vec{v}, \vec{\mu} \vDash \phi$$

The **time-store** operator $\overset{i}{\uparrow}$ writes the evaluation time into register v_i; the **time-recall** operator $\overset{i}{\downarrow}$ shifts the evaluation time to the stored value v_i. The **world-store** operator $\overset{i}{\Uparrow}$ writes the evaluation world into register μ_i; the **world-recall** operator $\overset{i}{\Downarrow}$ shifts the evaluation world to the stored value μ_i.

Remark **Notation**

The paper writes the four families as a single arrow pair subscripted by the coordinate being stored ($\overset{i}{\underset{\text{T}}{\uparrow}}/\overset{i}{\underset{\text{T}}{\downarrow}}$ for times, $\overset{i}{\underset{\text{M}}{\uparrow}}/\overset{i}{\underset{\text{M}}{\downarrow}}$ for worlds); this chapter uses single and double arrows instead, so that the coordinate is visible in the glyph.

These operators generalize Vlach's single now/then pair in two directions at once: the single pair becomes an indexed family, so that arbitrarily many times can be stored and recalled, and the mechanism is duplicated in the world coordinate, which Vlach's tense-anaphora setting did not require [22]. Apart from threading $\vec{v}$ and $\vec{\mu}$ through the point of evaluation, the semantics of the base language is unchanged: the truth conditions of Section 3.4 carry over verbatim, ignoring the new parameters.

10.3 Worked Evaluations

The register mechanics are best absorbed through small computations. Fix a model, a history τ, and a time x, and abbreviate the extended evaluation point as $(\tau, x, \vec{v}, \vec{\mu})$.

Store-then-recall collapses. Consider $\overset{0}{\uparrow} F \overset{0}{\downarrow} \phi$ ("store now; at some future time, recall and check ϕ"). Unfolding: the store writes x into v_0; the F shifts evaluation to some $y > x$; the recall shifts it back to $v_0 = x$. The whole formula is therefore true at (τ, x) iff ϕ is true at (τ, x) and some strictly future time exists – and seriality guarantees the latter, so

$\overset{0}{\uparrow} F \overset{0}{\downarrow} \phi$ is equivalent to ϕ. Storing a point and recalling it under a single shift undoes the shift.

Cross-referencing under a guard. Now interleave material between shift and recall: $\overset{0}{\uparrow} G\left(\phi \rightarrow \overset{0}{\downarrow} \psi\right)$ says "for every future time at which ϕ holds, ψ held back at the start". Unfolding gives: for all $y > x$, if ϕ at (τ, y) then ψ at (τ, x) – equivalent to $F\phi \rightarrow \psi$. In the world coordinate the same pattern reads $\overset{0}{\Uparrow} \Diamond\left(\phi \wedge \overset{0}{\Downarrow} \psi\right)$: "some possible world verifies ϕ while the **original** world verifies ψ", equivalent to $\psi \wedge \Diamond\phi$.

Each of these small patterns reduces to a register-free formula, which is precisely the wrong induction to draw from them: the reductions work because a single stored point is recalled under a single shift. Nested patterns that store several points and recall them across each other's scopes resist such flattening, and Cresswell's argument shows the situation is not remediable by fixing any finite register discipline in advance [23]: natural-language cross-reference eventually demands as much power as explicit quantification over times and worlds. The natural-language sentence of the opening section is the canonical illustration – its "now" must survive under a past-tense shift **and** interact with a quantifier's restrictor, which is exactly the combination the flattening tricks cannot reach.

10.4 The Stability Operator and BL*

The full extension tower also includes a restricted modality. Given a world $\tau \in H_{\mathcal{F}}$ and time $x : D$, let $\langle\tau\rangle_x := \{\sigma \in H_{\mathcal{F}} \mid \sigma(x) = \tau(x)\}$ be the set of possible worlds that intersect τ at x.

Definition 10.2 **Stability**

$\mathcal{M}, \tau, x \vDash \boxdot \phi \iff \mathcal{M}, \sigma, x \vDash \phi$ for all $\sigma \in \langle\tau\rangle_x$.

The **stability** operator $\boxdot$ quantifies over the worlds occupying the same world state as the world of evaluation at the time of evaluation. Since intersection-at-x is an equivalence relation on $H_{\mathcal{F}}$, the monomodal logic of $\boxdot$ is again S5; and for non-temporal ϕ, truth at $\langle\mathcal{M}, \tau, x\rangle$ depends only on the world state $\tau(x)$, so $\phi \rightarrow \boxdot\phi$ is valid on that fragment – stability collapses to the trivial modality on non-temporal formulas [1]. Letting $\diamondsuit\phi := \neg\boxdot\neg\phi$, the stability operators make *Will* and *Could* modals definable ($\boxdot G\phi$, $\boxdot F\phi$, $\diamondsuit G\phi$, $\diamondsuit F\phi$): what will or could unfold from the present world state. Restricting the intersection set forward or backward yields the open-future and open-past sets $|\tau\rangle_x := \{\sigma \in H_{\mathcal{F}} \mid \sigma(y) = \tau(y) \text{ for all } y \leq x\}$ and $\langle\tau|_x := \{\sigma \in H_{\mathcal{F}} \mid \sigma(y) = \tau(y) \text{ for all } y \geq x\}$, with corresponding operators quantifying over the worlds that share the evaluation world's past or future.

The philosophical payoff is that all three restricted quantifier domains are *definable* from the construction of possible worlds – provably $|\tau\rangle_x \subseteq \langle\tau\rangle_x$ and $\langle\tau|_x \subseteq \langle\tau\rangle_x$ with $|\tau\rangle_x \cap \langle\tau|_x = \{\tau\}$ – whereas a semantics over structureless points would have to posit primitive accessibility relations and then impose frame constraints to keep them aligned with their intended readings [1].

The language that collects all of these resources is

$$\mathrm{BL}^\star := \langle \mathrm{SL}, \bot, \to, \Box, S, U, \boxdot, \overset{i}{\uparrow}, \overset{i}{\downarrow}, \overset{i}{\Uparrow}, \overset{i}{\Downarrow} \rangle,$$

the terminus of the extension tower that begins with the tense basis and passes through the Until/Since basis. The paper explicitly declines to axiomatize $\mathrm{BL}^\star$ – extending **TM** to a proof system for it is outside that paper's scope [1] – and this book inherits the same boundary: no proof system for $\mathrm{BL}^\star$ is presented here, and none is formalized. The paper's one worked use of the Vlach operators is its analysis of the open future, for which see [1] and the closing remark of the semantics chapter on the determined/deterministic distinction.

10.5 Prior Art: From "Now" to Hybrid Binders

The store/recall pattern has a well-charted history, and $\mathrm{BL}^\star$ is best understood as its two-coordinate generalization.

- **Kamp 1971** introduced *now* as a rigid temporal reference point: however deeply embedded, *now* returns evaluation to the utterance time [7].
- **Vlach 1973** added *then*, making the reference point programmable: a storage operator marks a time, and *then* recalls it – the first genuine store/recall pair [22].
- **Cresswell 1990** argued that the general case requires unboundedly many stored indices: two, three, or any fixed number of registers eventually fails, so storage operators approach full explicit quantification over times [23].
- **Hybrid logic** systematizes the pattern with the $\downarrow$-binder, which names the current point, and the "at" operator, which returns evaluation to a named point [24].
- **Goranko's reference pointers** are the direct temporal-logic ancestor of the indexed-register formulation used here [25].

Against this backdrop, the Vlach families of $\mathrm{BL}^\star$ are hybrid-style binders over the time *and* world coordinates, with an indexed register vector in place of hybrid logic's named variables. That identification cuts both ways: the hybrid $\downarrow$-binder has a well-known cost profile – undecidability in general, with tamer behavior only in bounded or linearly-ordered fragments – and that profile is what makes the ceiling-and-descent

narrative of Chapter 11 the expected one for the BL* tower. No results are stated here; the frontier chapter surveys the published landscape.

10.6 Kamp's Theorem, Correctly Scoped

The deepest expressiveness fact in this neighborhood concerns the Until/Since basis itself, and it is worth stating with both of its scope conditions explicit.

Theorem 10.3 **Expressive Completeness (Kamp)**

Over Dedekind-complete flows of time, the temporal language with *Until* and *Since* in their strict readings is expressively complete for the first-order theory of linear order: every first-order condition on a linear order with monadic predicates, in one free variable, is equivalent to a temporal formula.[21]

Two conditions do real work. First, the operators must be *strict*: they quantify over strictly earlier and strictly later times, excluding the present. **TM** uses exactly this convention natively (Section 15.3), so the theorem speaks directly to the book's language rather than to a variant of it. Second, the flow of time must be *Dedekind complete*; over arbitrary linear orders the result fails, which is why the completeness property of the paper's frame-constraint hierarchy earns its keep. The theorem is frequently miscited to Kamp's 1971 *Theoria* paper; that paper introduces the *now* operator (its role in the prior-art narrative above) and does not contain the expressive-completeness theorem.

A completing note explains a difference from LTL that the next chapter revisits: over discrete, future-only, $\mathbb{N}$-like flows, the past-free fragment already suffices for first-order expressive completeness [28]. This is why LTL can afford to be a future-only language. **TM** keeps its past operators because its flows are arbitrary ordered abelian groups, where no such reduction is available.

10.7 The Formalization Frontier

A machine-checked Kamp theorem is an open problem. `Metalogic/WeakCanonical/Kamp/` develops the Rabinovich proof chain [27] toward it: the target statement `kamp_prior_expressive_completeness` says that every `MonadicFormula` with one free variable has an Until/Since-equivalent formula on Prior structures, with the translation implemented as `rabinovich_translate`. The end-to-end theorem remains open.

[21] Kamp's 1968 dissertation [26]; the modern model-theoretic proof is due to Rabinovich [27]. This is a paper-side theorem with no Lean anchor in this repository; see the formalization-frontier note below.

11 The Decidability Frontier

What expressive extensions of a temporal-modal logic cost, surveyed from published results: the linear-temporal floor, the product zone one modal dimension up, the cross-referencing ceiling, and the descents that linear flows and bounded disciplines buy back.

Dependencies: Chapter 10 for the store/recall operators; Chapter 7 for **TM**'s own decision procedure.

11.1 The Question

The two preceding chapters expanded the language twice: from the tense basis to Until/Since, and from there to the store/recall operators and the stability modality of BL*. Each expansion is well motivated, and each raises the same question: what does the added expressive power cost? For temporal-modal logics the currency is decidability and complexity of the satisfiability problem, and the published literature prices the neighborhood well enough to chart a frontier before any new result is stated.

11.2 The Floor and the Product Zone

The floor is the linear-temporal base itself: LTL satisfiability is PSPACE-complete [29] – hard, but comfortably decidable, and the benchmark every extension is measured against.

Adding one S5-like modal dimension moves into the territory of products of modal logics [11]. The bimodal products of a linear tense logic with S5 remain decidable, but the price is already steep: complexity for such products rises to EXPSPACE-hard territory [30], and the complete axiomatizations for knowledge-and-time logics in the same zone reflect the same jump [31]. The ceiling *within* the product landscape is sharp: three-dimensional products in the vicinity of S5 $\times$ S5 $\times$ S5 are undecidable and non-finitely-axiomatizable [32]. The published pattern, then, positions one modal dimension over linear time as the safe zone: expensive, but on the right side of the line.

11.3 The Cross-Referencing Ceiling – and the Linear Descent

Cross-referencing devices price differently, and higher. The hybrid binder $\downarrow$, which names the current evaluation point for later recall, makes satisfiability undecidable over arbitrary frames [33], [34]. But the ceiling has a descent: over *linear* structures, hybrid logics with binders become decidable again, at non-elementary cost [35]. Freeze quantifiers and registers tell the same story in a metric register: LTL with one freeze register over

the future is decidable but non-primitive-recursive, while two registers – or one register with past operators – tips into undecidability [36]; the clock discipline of timed temporal logic is the tamer alternative that keeps real-time reasoning decidable [37]. Trace quantification prices highest of all: HyperLTL satisfiability is undecidable [17], its model checking is decidable with cost rising by an exponential per quantifier alternation [38], and the logic embeds into a proper fragment of first-order logic with an equal-level predicate [39] – the line of work originating with [16]. Goranko's reference pointers, the direct ancestor of the store/recall formulation, come with their own expressiveness hierarchies [25].

The published pattern is consistent: unrestricted cross-referencing over rich frame classes costs decidability; linearity of the flow, bounded registers, anchoring disciplines, and quantifier restrictions buy it back at elevated complexity.

11.4 Where BL* Sits

BL*'s indexed store/recall families range over *both* times and worlds, giving the language hybrid-binder-like power in two coordinates simultaneously. By the pattern above, the expectation from prior art is clear: undecidability at the top of the tower, with decidable fragments only under register bounds, anchoring disciplines, or quantifier restrictions. That is an expectation, not a theorem – no result about the tower is stated or attributed here, and the paper's own boundary (no axiomatization of BL*, Chapter 10) applies with equal force to its computational theory.

11.5 Charting the Tower

A complexity map for the BL* tower would need to chart, at minimum: satisfiability for each register budget, in each coordinate separately and in both together; the effect of restricting recall to anchored (utterance-point) reference in the style of *now*; the interaction of the stability modality with the store/recall families; and which fragments inherit the linear-flow descent that [35] established for hybrid binders. Until such a map is published, the safe summary is the sectional pattern above: every cell of the map has a published analogue predicting its rough altitude, and none of the analogues predicts a free lunch.

11.6 Applications Outlook

The practical stakes mirror those of the model-checking tradition [9]: specification languages earn their keep when their verification problems are mechanizable, and every expressive extension that survives the frontier with decidability intact widens the class of systems whose temporal-

modal properties can be checked rather than argued. Cross-referencing operators are attractive for exactly the specifications that motivate them in natural language – properties relating the evolving present to a remembered reference point, or one possible evolution to another – and the frontier surveyed here determines when such specifications remain checkable.

11.7 TM's Own Position

TM itself sits below the frontier. The paper claims decidability via the finite model property [1]. In this repository, the tableau procedure's soundness is proven (`decide_sound`), and the finite-filtration FMP statement is sorry-free (`fmp_completeness`), with the semantic-validity bridge an open problem; Chapter 7 gives the precise statements. The expressive ascent charted in this part therefore starts from decidable ground, and the frontier above marks how far the ascent can go before that ground gives way.

PART II

Applications

Proof automation tactics and the bounded proof-search engine; the dual-signal training-data pipeline (proof traces to policy, countermodels to value, every output deterministically checkable); and dual-verification worked examples drawn from the `Examples/` library.

12 Proof Automation

The 4,300-line tactic, Aesop-integration, and bounded-proof-search half of `Automation/`*: what each entry point does, one worked invocation each, and a precise account of what is wired to what.*

Dependencies: Chapter 3 (proof theory) for the axiom/rule vocabulary these tactics target.

12.1 Tactics

Three user-facing tactics automate common derivation patterns; `apply_axiom` and `modal_t` live in `Automation/Tactics/Helpers.lean`, and `tm_auto` in `Tactics/Commands.lean`.

– `apply_axiom` (`Tactics/Helpers.lean:96`) – expands to `apply DerivationTree.axiom; refine ?_`, unifying the goal with an axiom schema and letting Lean infer the axiom's formula parameters via `refine`.

– `modal_t` (`Tactics/Helpers.lean:113`) – named for the T axiom $\Box\varphi \to \varphi$; its macro body expands identically to `apply_axiom`'s (`apply DerivationTree.axiom; refine ?_`), so it applies to any axiom-shaped goal.

– `tm_auto` (`Tactics/Commands.lean:284`) – delegates to `runModalSearch` with `SearchConfig.default` (search depth 10, overridable: `tm_auto 5`). It is implemented directly over the bounded search engine below, not over Aesop.

A typical invocation: given a goal of the shape "$\Box\varphi \to \Box\Box\varphi$" (the M4 pattern), `tm_auto` searches up to depth 10 via bounded proof search (Section 12.3 below) and closes the goal if a derivation exists within that bound. Related tactics `modal_search` (`Tactics/Commands.lean:105`), `temporal_search` (`Tactics/Commands.lean:177`), and `propositional_search` (`Tactics/Commands.lean:233`) restrict the search to a single operator family.

12.2 Aesop Integration

`AesopRules.lean` (276 lines, sorry-free) registers `@[aesop safe apply]`/`@[aesop safe forward]`/`@[aesop norm unfold]` rules over direct axiom applications (7 rules: `modal_t`, `prop_k`, `prop_s`, `modal_4`, `modal_b`, `temp_4`, `temp_a`), forward-chaining variants of the same seven, three inference-rule apply rules (modus ponens, modal K, temporal K), and four normalization unfold rules. Excluded by design (soundness incomplete for these under the current axiom set): `temp_l`, `modal_future`, `temp_future_derived`.

Every Aesop attribute in the file is unqualified, so the rules register into Aesop's **default** rule set; there is no dedicated rule set for the logic. `AesopRules.lean` serves direct `aesop` invocations by users who opt in explicitly – `tm_auto` itself does not route through Aesop. Reach for plain `aesop` (default rule set) when a goal is a short chain of the seven covered axioms; reach for `tm_auto` for anything requiring the bounded search engine's heuristics.

12.3 Bounded Proof Search

`ProofSearch/Core.lean` (1,195 lines, sorry-free) is the search engine `tm_auto` and its variants call into. `bounded_search` (`ProofSearch/Core.lean:958`) is a depth-limited, memoized DFS (default visit limit 500) with several heuristic-ordering functions (`heuristic_score` at `ProofSearch/Core.lean:829`, `advanced_heuristic_score` at `ProofSearch/Core.lean:865`, `pattern_aware_score` at `ProofSearch/Core.lean:914`) that reorder subgoals to prefer promising branches; `bounded_search_with_proof` (`ProofSearch/Core.lean:1064`) is the proof-carrying variant that actually produces a `DerivationTree`. `iddfs_search` (`ProofSearch/Core.lean:1168`, default max depth 100, default visit limit 10000) iteratively deepens `bounded_search`, and is documented as complete and optimal (shortest proof) within the max-depth bound. `ProofSearch/Strategies.lean` (379 lines, sorry-free) adds a priority-queue best-first search (`bestFirst_search`, `ProofSearch/Strategies.lean:90`) and a `SearchStrategy` dispatcher (`ProofSearch/Strategies.lean:187`: `BoundedDFS`/`IDDFS`/`BestFirst`, default IDDFS depth 100) that `search` (`ProofSearch/Strategies.lean:219`) selects between, plus a learning variant (`search_with_learning`, `ProofSearch/Strategies.lean:304`) that records success patterns (below) across calls. Relationship to the tableau (Chapter 7): this is a **different** algorithm on the **same** underlying proof system – the tableau builds a refutation tree over signed subformulas of a single target formula and is what `decide` ultimately falls back to, while this search engine explores `DerivationTree` construction directly and is what the tactics above call; `decide` itself also tries this engine first, as a fast path, before falling back to the tableau (Chapter 7).

12.3.1 The Search Space

A search node is a pair of a context Γ and a goal formula ϕ – exactly the data of a derivability claim $\Gamma \vdash \phi$ – and the engine explores backward from the target claim toward closed leaves. At each node, `bounded_search` proceeds through an ordered cascade:

1. **Leaf checks first.** If ϕ matches an axiom schema (`matches_axiom`) or is an assumption in Γ, the node closes immediately.

2. **Backward modus ponens.** Otherwise the engine collects every ψ such that $\psi \to \phi$ is available from Γ (`find_implications_to`) and recursively searches each antecedent ψ at depth $d - 1$, cheapest-scoring antecedent first.
3. **Structural descent.** If the goal is itself modal or temporal ($\Box\psi$ or an Until formula), the engine descends into ψ under the correspondingly transformed context – the backward image of the necessitation-style rules.

Three mechanisms keep the exploration tractable. A **memoization cache** keyed on (Γ, ϕ) pairs stores completed verdicts, so the same claim is never searched twice; a **visited set** on the current path blocks cycles; and a global **visit limit** (default 500) bounds total work even when the depth bound alone would admit an exponential frontier. The engine also accumulates `SearchStats` (visits, cache hits and misses, limit-pruned branches), which the learning layer below consumes.

12.3.2 Heuristic Ordering

The subgoal ordering is where the engine's domain knowledge lives. `heuristic_score` assigns each candidate a cost: axiom-shaped goals score lowest, context assumptions next, modus-ponens candidates score by the complexity of their cheapest antecedent, and modal or temporal goals pay a base cost plus a penalty growing with context size; goals with none of these prospects score as dead ends. `advanced_heuristic_score` layers domain-specific adjustments on top – bonuses for modal and temporal goals and a damped structural penalty `structure_heuristic` combining formula complexity, modal depth, temporal depth, and implication count – and `pattern_aware_score` further adds bonuses from the learned pattern database described below. All weights are configurable through a `HeuristicWeights` record, so the default ordering can be retuned without touching the algorithm.

12.3.3 Worked Invocations

Consider the M4 pattern $\Box p \to \Box\Box p$ under `tm_auto`. The goal is not an assumption; `matches_axiom` recognizes it against the axiom table (M4 is primitive in BX), and the search closes at depth 1 with a one-node `DerivationTree`. A goal requiring genuine search, such as a chained implication whose antecedents must themselves be derived, exercises the modus-ponens cascade: each candidate antecedent is scored, the cheapest is searched first, and the memoization cache ensures that shared antecedents across branches are proven once. The single-family variants behave identically but restrict the candidate moves: `modal_search` closes $\vdash \Box p \to p$ (T) and $\vdash \Box p \to \Box\Box p$ (4) without considering temporal rules, which shrinks the branching factor when the goal's operator family

is known in advance (worked instances in the `Examples/` library are collected in the dual-verification chapter).

12.4 Learning and Game-Theoretic Tactics

`SuccessPatterns.lean` (423 lines, sorry-free) implements a `PatternDatabase`-based heuristic layer: it records structural features (modal depth, temporal depth, top-level operator, context size) of goals that were successfully closed, and boosts the heuristic score of future goals matching those patterns, citing the machine-learning-guided-search literature (Yang et al. 2019; Kaliszyk et al. 2018) as its design inspiration. The loop is deliberately conservative: patterns only ever **reorder** the search frontier via `pattern_aware_score`, so a mistrained database can slow the search but can never cause an unsound answer – soundness lives entirely in the `DerivationTree` terms the search produces, never in the heuristics that find them. `EFGameTactics.lean` (326 lines, sorry-free) is a narrower-purpose module: tactic macros (`simp_game_tuple`, `game_tuple_unfold`, and per its header comment, components for pivot-order and winning-condition reasoning) automating repetitive steps in the `WeakCanonical/EFGames` Ehrenfeucht-Fraïssé game infrastructure, built specifically for the Gabbay-Hodkinson-Reynolds expressive-completeness proof technique [40]. The module belongs to the discrete-case expressiveness infrastructure of the meta-logic chapter: the GHR EF-game technique is the classical descendant of Kamp-style expressive-completeness arguments [7] for temporal logic over linear orders, and `EFGameTactics.lean` automates the game-position bookkeeping that technique requires.

12.5 Module Map

Module	Lines	Role
`Tactics/Helpers.lean`	1,032	`apply_axiom`, `modal_t`, and supporting macro infrastructure
`Tactics/Commands.lean`	710	`tm_auto`, `modal_search`, `temporal_search`, `propositional_search`
`ProofSearch/Core.lean`	1,195	`bounded_search`, `iddfs_search`, heuristic scoring, memoization
`ProofSearch/Strategies.lean`	379	Best-first search, `SearchStrategy` dispatcher, learning variant
`AesopRules.lean`	276	Aesop rule registrations for direct `aesop` use
`SuccessPatterns.lean`	423	`PatternDatabase` learned-heuristic layer
`EFGameTactics.lean`	326	EF-game bookkeeping tactics for `WeakCanonical/EFGames`

Table 24: The tactic and proof-search half of `Automation/` (live line counts). The dataset-pipeline half is covered in Chapter 13.

All modules in the table are sorry-free.

13 The BMLogic Dataset Pipeline

The dual-signal training-data pipeline that turns the formal decision procedure into machine-learning-ready datasets – architecture, module map, the Tier-1 feasibility gate results (including the provability-ratio shortfall), and the Tier-2 response.

Dependencies: Chapter 2 (decidability, Chapter 7) for `decide` and its certificate/countermodel outputs.

13.1 The Practitioner Thesis, Restated

Decidable fragments (Chapter 7) are a target for full automation; the full logic is a target for training AI systems to reason proof-theoretically, because every enumerated formula's decision yields a **deterministically checkable** training signal – a kernel-checked proof term or an explicit countermodel, never a merely-plausible label. This chapter is the canon-

ical narrative home of that pipeline; `docs/training/PIPELINE.md` holds the operational quick-reference.

13.2 Dual-Signal Architecture

For each enumerated formula, the decision procedure (`decide`, Chapter 7) yields exactly one of two supervisory signals:

- **Positive signal – proof traces (policy network)**: for a formula decided valid, a `ProofTrace` records derivation `height`, the `axioms_used`, and the `rules_applied`. "The policy network learns to predict **which axioms and rules to apply** at each step of a proof search."[22]
- **Corrective signal – countermodels (value network)**: for a formula decided invalid, a `SimpleCountermodel` records `trueAtoms`/`falseAtoms`/`formula`. "The value network learns to estimate the probability that a given proof state leads to a valid proof… they teach the network which formula shapes are **not** theorems."[23]
- **Enriched corrective signal (Tier 2)**: `EnrichedCountermodel.lean` retains the full saturated tableau branch (which modal/temporal subformulas held or failed); the module is implemented and tested, and its integration into the main export path belongs to the pipeline's second tier.[24]

13.3 Pipeline Flow and Module Map

`FormulaEnumerator.lean` enumerates formulas up to a depth/size bound; `DatasetGenerator.lean` labels each via `decide`, extracting a proof trace or countermodel; `DatasetExport.lean` (JSONL, feeding the `dataset_generator` executable) and `DatasetExporter.lean` (structured JSON, feeding a Python tensor-conversion script) export the labeled dataset. The pipeline comprises seven Lean modules – `DataExport.lean`, `FormulaEnumerator.lean`, `DatasetGenerator.lean`, `EnrichedCountermodel.lean`, `DatasetExporter.lean`, `DatasetExport.lean`, and `DatasetValidator.lean` – with `DataExport.lean` serving as a shared dependency of the other six rather than a pipeline stage in its own right.[25]

---

[22] `docs/training/PIPELINE.md:24-31`.

[23] `docs/training/PIPELINE.md:33-40`.

[24] `docs/training/PIPELINE.md:42-44`; `Automation/EnrichedCountermodel.lean` (211 lines per `Automation/README.md`).

[25] `docs/training/PIPELINE.md`, Module Reference section.

13.3.1 Anatomy of a Dataset Record

Each exported JSONL line is a `DatasetRecord` (`Automation/DatasetExport.lean:130`), carrying the formula in several parallel encodings alongside its label and exactly one supervisory payload. A representative valid-formula record, abridged from the schema documented at the head of `DatasetExport.lean`:

```
{
  "id": "bmlogic-00001",
  "split": "train",
  "formula_str": "(□p → p)",
  "formula_ast": {"tag": "imp", "left": {"tag": "box", ...},
"right": ...},
  "frame_class": "Base",
  "label": "valid",
  "proof_trace": {"height": 0, "axioms_used": ["modal_t"],
"rules_applied": []},
  "countermodel": null,
  "pattern_key": {"modalDepth": 1, ...},
  "metrics": {"complexity": 3, ...}
}
```

The record design encodes the dual-signal contract structurally: `proof_trace` (a `ProofTrace` of derivation `height`, `axioms_used`, `rules_applied`) is populated exactly when the label is `valid`, and `countermodel` (the `SimpleCountermodel` triple of `trueAtoms`/`falseAtoms`/`formula`) exactly when it is `invalid`. The redundant formula encodings serve different consumers: `formula_str` for human inspection, `formula_ast` (the `Formula.toJson` tag schema) for tokenizer-free tree models, an S-expression and a prefix-notation token list for sequence models, and folded variants that restore derived-operator vocabulary ($\neg$, $\wedge$, $\vee$, $\Diamond$) for models trained on the surface language. `pattern_key` mirrors the structural features the proof-search learning layer uses (Chapter 12), and `metrics` records difficulty measures (complexity, modal depth, temporal depth) that support curriculum ordering and stratified splits downstream.

Two `lake exe` executables compile from this pipeline:[26]

– `dataset_generator` (root `Bimodal.Automation.DatasetExport`) – the main JSONL export executable.

– `dataset_validator` (root `Bimodal.Automation.DatasetValidator`) – validates exported datasets against the schema contract.

[26] `docs/training/PIPELINE.md:428-437`, quoting the `lakefile.lean` executable declarations.

13.4 BimodalHarness Integration: Artifact-Only

"The integration between the two repositories is **artifact-only**: BimodalHarness never calls Lean at runtime. Instead, it reads JSONL files exported by `lake exe dataset_generator` and synced via a sync command."[27] The sync mechanism is a plain file-sync step: generate JSONL in BimodalLogic, a sync command copies `data/` to the BimodalHarness data directory, and BimodalHarness's Python side reads the synced files – no cross-repository Lean dependency at training or inference time.

13.5 The Tier-1 Feasibility Gate

A small-config gate (modal depth 2, temporal depth 2, max size 8, 3 atoms) ran 30/30 conformance tests successfully and produced 254,252 distinct formulas, but the feasibility criteria were **not** all met:

Criterion	Target	Actual	Result
Distinct formulas	$\geq$ 1,000	254,252	PASS
Provability ratio	0.15–0.70	0.033 (3.2% valid)	FAIL
Proof height variance	> 2.0	0.0	FAIL
≥ 3 categories $> 10\%$	≥ 3	4	PASS
$< 80\%$ trivially propositional	$< 80\%$	35.9%	PASS
$< 90\%$ same decision	$< 90\%$	92.6%	FAIL

Table 25: Tier-1 feasibility gate results (`docs/training/PIPELINE.md:687-740`): overall gate decision **FAILED**, 3 of 6 hard criteria not met.

This is the "sizable remaining project" the introduction's practitioner thesis points to: the raw enumeration over-produces non-theorems (92.6% invalid, only 3.2% valid against a 15% minimum target), so a naive dataset is imbalanced and low-variance on the policy side.

13.6 The Tier-2 Response: Theorem Mining

The recommended response is **not** to abandon the dual-signal approach but to change how formulas are sourced for labeling: "Generate formulas by composing known axiom instances (apply modus ponens closure to BX axiom schemata). This guarantees new valid formulas at higher complexity."[28] Complementary approaches noted in the same source:

[27] `README.md:183-184` and `docs/training/PIPELINE.md:14`, near-identical wording in both places; sync mechanism detailed at `docs/training/PIPELINE.md:612`.

[28] `docs/training/PIPELINE.md:744-762`, "Priority 1: Address Provability Ratio (Gate Blocker)."

biased enumeration (a two-pass valid-template-plus-random-padding strategy) and restricting to smaller formula sizes. The pipeline's second tier therefore comprises two components: the theorem-mining generator, sourcing valid formulas by axiom composition as above, and the wiring of `EnrichedCountermodel.lean` into the main export path, activating the richer corrective signal noted earlier. `EnrichedCountermodel.lean` itself is implemented and tested; the generator and the wiring constitute the second tier's engineering scope.

13.7 Shipped Machine-Readable Axiomatization

The axiomatization itself – every axiom schema, inference rule, and derived-operator definition – is shipped with this book as a machine-readable JSONL artifact at `Theories/Bimodal/typst/generated/machine-appendix.jsonl`, rendered as tables in the back-matter appendix (*Appendix: The Machine-Readable Axiomatization*). The formula encoding is the same `Formula.toJson` tag schema emitted by `dataset_generator`, so the shipped axiomatization and the training datasets above share one encoding. The artifact is regenerated commit-stamped via `scripts/typst-machine-appendix.sh` (which wraps `lake exe machine_appendix`, extracting each schema from the `Axiom` type index rather than transcribing it), and `scripts/typst-sync-check.sh` enforces its freshness against live Lean source.

13.8 Operational Pointer

`docs/training/PIPELINE.md` holds the build/run commands and configuration knobs; this chapter is the canonical architecture narrative.

14 Dual Verification and Worked Examples

The dual-verification architecture (proof certificates vs. countermodel search) framing this book's worked material, plus curated derivations from the sorry-free `Examples/` library rendered dual-track.

Dependencies: Chapters 3–5 (proof theory, metalogic, theorems); Part II's frame-classes and decidability chapters for the semantic side.

14.1 Dual Verification

"Together with ProofChecker, [ModelChecker] forms the dual verification architecture: ModelChecker searches for countermodels while

[29] `README.md:183`.

ProofChecker constructs formal derivations."[29] Adapting the Logos manual's framing:[30]

– **Proof certificates** – a kernel-checked `DerivationTree` term is a machine-verifiable witness explaining **why** a formula is a theorem, not merely **that** it is one. Every `valid` result from `decide` (Chapter 7) carries one.

– **Counterexamples** – an explicit `SimpleCountermodel` (or the richer `SemanticCountermodel`) gives targeted corrective feedback: not merely that an inference fails, but which atoms make it fail.

– **Soundness guarantees** – soundness (proven for `TM`, all three frame classes) is what makes a proof certificate reliable independent of the search procedure that found it: if the kernel accepts a `DerivationTree`, the corresponding formula is genuinely valid.

In this repository, the lean-verified instance of this architecture is the decision procedure of Chapter 7 (`decide_sound` proven, countermodels extracted from open tableau branches); the cross-project vision above remains architectural only, where ModelChecker's independent Z3-based search would corroborate ProofChecker's Lean-checked derivations for the same formula.

14.2 The Workflow, End to End

The dual-verification loop for a candidate formula ϕ has a single entry point and two exits.

1. **Pose.** Formulate the conjecture as a `Formula` term – say the P1 pattern $\Box p \to \triangle p$ or its converse $\triangle p \to \Box p$.
2. **Decide.** Run `decide` (or, inside a proof, attempt `tm_auto`/`modal_search`). The procedure tries the fast paths first – direct axiom match, bounded proof search – and falls back to the tableau.
3. **Exit valid.** The result carries a `DerivationTree` proof term. Lean's kernel re-checks the term independently of the search that found it, so acceptance does not depend on trusting the decision procedure – this is the content of `decide_sound`.
4. **Exit invalid.** The result carries a `SimpleCountermodel` naming the atom assignment that breaks the formula, extracted from an open saturated tableau branch. The countermodel is a checkable artifact in its own right: evaluating the formula under the reported assignment reproduces the failure.

The two conjectures above illustrate the exits. The P1 pattern closes as a theorem (`perpetuity_1`, below). Its converse fails, and the shape of the failure is instructive: a formula can hold at every time along one

[30] Logos `01-introduction.typ`, "Dual Verification" section (external repository, commit-pinned by citation; not a local Lean name).

history while failing on another history through a different world state, so "always" does not imply "necessarily" – the semantic content of the countermodel any run on the converse pattern produces. Each exit hands downstream consumers a certificate of the corresponding kind, which is precisely the dual-signal contract the dataset pipeline of Chapter 13 packages at scale.

14.3 Worked Examples: Perpetuity in Practice

`Examples/BimodalProofs.lean` (241 lines, sorry-free) demonstrates the perpetuity principles (Chapter 15) as concrete, checked derivations rather than abstract statements.

Example **P1 Applied to an Atom**

`⊢ φ.box.imp φ.always := perpetuity_1 φ`[31] instantiates immediately to any formula; concretely, for an atom: `⊢ (Formula.atom_s "p").box.imp (△(Formula.atom_s "p")) := perpetuity_1 _`[32] – necessity of p implies p holds always, both notations (`.always` and $\triangle$) proven definitionally equal.

Example **P5: Persistent Possibility**

`⊢ φ.sometimes.diamond.imp φ.diamond.always := perpetuity_5 φ`[33] – if φ is possibly true at some time, then it is always possible that φ – the most semantically rich of the six perpetuity principles, connecting temporal existence with modal persistence.

Example **Automated S5 Derivations via `modal_search`**

The T and 4 axioms, proven automatically rather than by direct axiom application:

```
example : ⊢ (Formula.atom_s "p").box.imp (Formula.atom_s
"p") := by modal_search
example : ⊢ (Formula.atom_s "p").box.imp (Formula.atom_s
"p").box.box := by modal_search
```

[31] `Examples/BimodalProofs.lean:52`.

[32] `Examples/BimodalProofs.lean:61`.

[33] `Examples/BimodalProofs.lean:124` (`noncomputable`, since the underlying modal-collapse argument is classical).

[34] `Examples/BimodalProofs.lean:211-217`; the proof-automation engine of Chapter 12 closes both goals within its default search depth.

[35] Via `all_future`; note a commented-out BX1/reflexivity test at `Examples/BimodalProofs.lean:219` documents that the T-axiom-style test for the **temporal**

[34] A capstone combined example in the same file (`Examples/BimodalProofs.lean:223`) shows $\Box$ distributing over the derived always-future operator, $\Box\varphi \to \Box(G\varphi)$[35], proven the same way.

14.4 Worked Examples: Concrete Temporal Structures

`Examples/TemporalStructures.lean` (277 lines, sorry-free) instantiates the abstract task-frame semantics of Section 3.4 over concrete duration types. The file concretely instantiates the **discrete** case – `intTimeFrame` (`Examples/TemporalStructures.lean:65`) and `intNatFrame` (`Examples/TemporalStructures.lean:78`) over `Int` – alongside a fully polymorphic `genericTimeFrame` (`Examples/TemporalStructures.lean:144`) and `genericNatFrame` (`Examples/TemporalStructures.lean:156`), generic over any ordered abelian group `D`; a concrete dense ($\mathbb{Q}/\mathbb{R}$) instantiation requires additional Mathlib imports and is not included. Sanity-check lemmas (`Examples/TemporalStructures.lean:217` and `Examples/TemporalStructures.lean:222`) confirm the generic frame specializes correctly to the concrete `Int` frame by `rfl`; `int_nullity_example` (`Examples/TemporalStructures.lean:229`) / `generic_nullity_example` (`Examples/TemporalStructures.lean:235`) and `int_compositionality_example` (`Examples/TemporalStructures.lean:242`) / `generic_compositionality` (`Examples/TemporalStructures.lean:257`) exercise the two task-frame axioms of Section 3.4 concretely.

14.5 Reproducibility Note

Every derivation cited in this chapter is sorry-free and can be rechecked by compiling `Theories/Bimodal/Examples/BimodalProofs.lean` and `TemporalStructures.lean` directly; no external tooling (ModelChecker, Z3) is required for the worked examples above.

15 Notes

15.1 Implementation Status

The syntax (six primitive constructors over the Until/Since basis), the task-frame semantics with strict truth conditions, and the BX proof system (42 axiom constructors in eight layers, 7 inference rules, frame-class parameter) are complete in Lean. Soundness for all three frame classes, the deduction theorem, and the perpetuity principles P1–P6 are

operator was intentionally disabled under the strict/irreflexive semantics convention (Section 15.3) – reflexive temporal T-axioms are not valid here.

proven. A canonical-model construction toward completeness is developed via `completeness` (`Metalogic/BXCanonical/`), with completeness itself remaining an open problem; the tableau decision procedure has soundness proven (`decide_sound`), and its finite-model-property component is discussed below.

15.2 Relation to the Published Presentation

This section documents differences between the paper [1] and the Lean implementation. The Lean source is authoritative for all implementation claims; the paper is authoritative for the informal narrative and target vocabulary.

15.2.1 Terminology

- The paper uses "perpetuity principles" for P1–P6; the Lean code uses the same terminology.
- The paper's notation $\triangle$ and $\triangledown$ for "always" and "sometimes" is preserved in the Lean implementation as `always` and `sometimes`.
- The paper's **Reflection** constraint on task frames is the Lean field `converse` (in biconditional form); the paper's **Nullity** is strengthened to the Lean field `nullity_identity`.

15.2.2 Language Basis

The paper's base language $\mathcal{L}$ takes H and G as primitive tense operators, and extends to $\mathcal{L}^+$ with Until and Since; the corresponding proof systems are **TM** and **TM**$^+$, related by a conservative-extension theorem. The Lean formalization works in the Until/Since basis throughout: `untl` and `snce` are primitive, $H/G/F/P$ are derived (see Section 2.1).

The module `Metalogic/ConservativeExtension/` contains `lift_derivation_qfree` (`Metalogic/ConservativeExtension/Lifting.lean`), a Goldblatt/BdRV-style fresh-atom naming lemma supporting the irreflexivity argument of Section 15.3 (see Section 5.6 in the Frame Classes chapter). It is distinct from the paper's $\mathcal{L}$-vs-$\mathcal{L}^+$ conservative-extension theorem, which is a paper-side result: no Lean module formalizes it.

15.2.3 Axiom Correspondence

The paper presents **TM** with twelve schemata; the Lean BX system has 42 constructors (granularity, not extra strength — see the proof-theory chapter). The paper's schemata map to Lean as follows:

Paper Name	Lean Counterpart	Status in BX
MP	`DerivationTree.modus_ponens`	Inference rule
MN	`DerivationTree.necessitation`	Inference rule
TD	`DerivationTree.temporal_duality`	Inference rule
MK	`Axiom.modal_k_dist`	Axiom
MT	`Axiom.modal_t`	Axiom
M5	`Axiom.modal_5_collapse`	Axiom
MF	`Axiom.modal_future`	Axiom
TK	`temp_k_dist_derived`	Derived theorem
T4	`temp_4_derived`	Derived theorem
TB	`Axiom.serial_future`	Axiom
TA	`Axiom.connect_future`	Axiom
TL	`Axiom.temp_linearity`	Axiom

The Lean system additionally includes M4 (`Axiom.modal_4`) and MB (`Axiom.modal_b`) as primitive S5 axioms (derivable from the paper's core but convenient in Hilbert-style derivations), the full Burgess-Xu Until/Since layer with primed past mirrors, and the frame-class layers (uniformity, Prior, Z1, density) that the paper treats as extensions of **TM**. TF ($\Box\phi \to G\Box\phi$) is derived as `temp_future_derived` (`Theorems/Combinators.lean`) from MF, MT, and M4.

15.2.4 Completeness Status

The paper [1] sketches completeness for **TM** and its extensions. In Lean the completeness theorems are stated for each frame class (`completeness`, `completeness_dense`, `completeness_discrete` in `Metalogic/BXCanonical/Completeness.lean`) and approached through the canonical-model construction; the open steps lie on the construction path (chronicle construction for the dense case, discrete truth lemma and transfer), and completeness remains an open problem. Soundness, the deduction theorem, the Lindenbaum lemma, and the perpetuity principles are fully proven. The discrete case runs through Kamp-theorem expressiveness [7] and likewise remains open.

15.2.5 Decidability Implementation

The implementation includes a tableau-based decision procedure for validity. Soundness is proven (`decide_sound`): if the procedure returns "valid", the formula is semantically valid. The completeness direction rests on the finite model property, developed in `Metalogic/Decidability/FMP/`: the module is sorry-free (see the Part II "Decidabil-

ity in Practice" chapter), but its `fmp_completeness` theorem quantifies over a finite closure-MCS-bundle filtration rather than directly over semantic validity, and the bridge from "true in every closure MCS bundle" to full semantic validity is an open problem.

15.3 Design Choices

15.3.1 Strict (Irreflexive) Temporal Semantics

The temporal operators G and H can be interpreted with either **strict** or **reflexive** quantification over times. **TM** uses strict semantics — the "A2 guard convention", documented in `Semantics/Truth.lean`:

Definition 15.1 **Strict Temporal Semantics (Current)**

Temporal quantification excludes the present moment:

$$\mathcal{M}, \tau, x \vDash G\phi \Leftrightarrow \mathcal{M}, \tau, y \vDash \phi \text{ for all } y : D \text{ where } x < y$$

$$\mathcal{M}, \tau, x \vDash H\phi \Leftrightarrow \mathcal{M}, \tau, y \vDash \phi \text{ for all } y : D \text{ where } y < x$$

Until and Since use a strict witness with an open guard. The temporal T-axioms $G\phi \to \phi$ and $H\phi \to \phi$ are **invalid**; seriality is supplied axiomatically (BX1/BX1′). This matches the truth conditions in Section 3.4 and the paper's semantics, which quantifies tense operators strictly.

Under the alternative reflexive convention ($\leq/\geq$), the temporal T-axioms are definitionally valid, the frame-class axioms trivialize, and the completeness architecture collapses to a single theorem; the strict convention was adopted together with the Burgess-Xu axiom system precisely to preserve these distinctions. The paper likewise distinguishes the irreflexive tense operators (primitive) from their reflexive companions (definable, but not conversely).

15.3.2 Consequences of Strict Semantics

- **Frame definability is real**: density ($GG\phi \to G\phi$), discreteness (Prior/Z1), and seriality genuinely characterize frame classes, which is what makes the `Base`/`Dense`/`Discrete` frame-class parameter of the proof system meaningful. Under reflexive semantics all of these collapse to trivial validity.
- **Three completeness targets**: the base, dense, and discrete systems each get their own completeness statement (`completeness`, `completeness_dense`, `completeness_discrete`), matching the paper's **TM**$^+$ extensions.
- **Irreflexivity is not modally definable** [41]: no axiom forces the canonical accessibility to be irreflexive. The construction compensates with fresh-atom machinery — the structured `Atom` type exists precisely so that a fresh atom is available outside any finite set — and

with the chronicle/transfer constructions of the metalogic chapter rather than a naive canonical model.

- **Seriality is axiomatic, not automatic**: BX1/BX1′ ($\top \to F\top$, $\top \to P\top$) require every time to have a strict successor and predecessor time, which holds in every nontrivial ordered abelian group of durations.

15.3.3 Historical Context

Remark **Prior's Tradition**

Arthur Prior established tense logic using **strict** semantics: F and P quantify over strictly future and strictly past times, and temporal axioms genuinely characterize frame properties. This tradition continues through Burgess [2], [3], Xu [4], Goldblatt, van Benthem, and Blackburn-de Rijke-Venema [41], and the BX axiom system places the implementation squarely within it.

Remark **Computer Science Conventions**

Model checking traditions often use reflexive conventions (CTL's "AG ϕ" includes the current state), trading frame-theoretic expressiveness for simpler boundary conditions. **TM** takes the opposite trade: reasoning about contingency across dense and discrete temporal orders requires the frame distinctions that only strict semantics can express.

Remark **The Reflexive Alternative**

A reflexive convention ($\leq$) yields a single collapsed completeness target in which the frame classes are indistinguishable; the strict convention ($<$, the A2 guard convention) supports the Burgess-Xu axioms, replaces the temporal T-axioms with seriality axioms, and sustains three genuinely distinct frame classes.

Appendix: The Machine-Readable Axiomatization

This appendix ships the complete **TM** axiomatization in machine-readable form: the 42 axiom schemata, the 7 inference rules of `DerivationTree`, and the 21 derived-operator definitions. The raw artifact is a JSONL file committed alongside this book at `Theories/Bimodal/typst/generated/machine-appendix.jsonl`; the tables below are **rendered from that artifact** (via `scripts/typst-machine-appendix.sh`), never hand-copied. It is intended for the AI-practitioner audience of Chapter 13: an agent or pipeline can consume the axiomatization directly, in the same formula encoding as the training datasets.

Each JSONL line is one JSON object tagged by a `kind` field: one `metadata` line (generator, version, commit stamps, counts), then one line per axiom (`kind: "axiom"`), inference rule (`kind: "inference_rule"`), and derived operator (`kind: "derived_operator"`). Axiom lines carry the constructor `name`, source `layer`, schematic `params`, minimum `frame_class` (`Base`, `Dense`, or `Discrete`, per `Axiom.minFrameClass`), and the schema formula both as a display string (`schema_string`) and as a structured tree (`schema`). Schemas and definitions are given over the six-constructor primitive basis, in the `Formula.toJson` tag encoding shared with the dataset pipeline:

Constructor (`tag`)	JSON shape
`atom`	`{"tag": "atom", "name": "<base>"}`
`bot`	`{"tag": "bot"}`
`imp`	`{"tag": "imp", "left": <φ>, "right": <ψ>}`
`box`	`{"tag": "box", "child": <φ>}`
`untl`	`{"tag": "untl", "event": <φ>, "guard": <ψ>}`
`snce`	`{"tag": "snce", "event": <φ>, "guard": <ψ>}`

Table 27: The `Formula.toJson` tag encoding (`Automation/DataExport.lean`), shared by the machine appendix and the dataset pipeline.

Loading the artifact is two lines of Python:

```
import json
records = [json.loads(line) for line in open("machine-appendix.jsonl")]
```

Derived operators are exported as **kernel-computed unfoldings**: each definition below is the real Lean `def` applied to schematic atoms, so the right-hand sides are the exact primitive-basis formulas the proof system manipulates. Schematic metavariables (φ, ψ, χ, ϑ, p) are encoded as atoms with those base names.

The Axiom Schemata (42 constructors, `ProofSystem/Axioms.lean`)

Name	Layer	Params	Class	Schema (primitive basis)
`prop_k`	Propositional	φ, ψ, χ	Base	`((φ → (ψ → χ)) → ((φ → ψ) → (φ → χ)))`
`prop_s`	Propositional	φ, ψ	Base	`(φ → (ψ → φ))`
`ex_falso`	Propositional	φ	Base	`(⊥ → φ)`
`peirce`	Propositional	φ, ψ	Base	`(((φ → ψ) → φ) → φ)`
`modal_t`	S5 Modal	φ	Base	`(□φ → φ)`
`modal_4`	S5 Modal	φ	Base	`(□φ → □□φ)`
`modal_b`	S5 Modal	φ	Base	`(φ → □(□(φ → ⊥) → ⊥))`
`modal_5_collapse`	S5 Modal	φ	Base	`((□(□φ → ⊥) → ⊥) → □φ)`
`modal_k_dist`	S5 Modal	φ, ψ	Base	`(□(φ → ψ) → (□φ → □ψ))`
`serial_future`	BX Temporal		Base	`((⊥ → ⊥) → U((⊥ → ⊥), (⊥ → ⊥)))`
`serial_past`	BX Temporal		Base	`((⊥ → ⊥) → S((⊥ → ⊥), (⊥ → ⊥)))`
`left_mono_until_G`	BX Temporal	φ, χ, ψ	Base	`((U(((φ → χ) → ⊥), (⊥ → ⊥)) → ⊥) → (U(ψ, φ) → U(ψ, χ)))`
`left_mono_since_H`	BX Temporal	φ, χ, ψ	Base	`((S(((φ → χ) → ⊥), (⊥ → ⊥)) → ⊥) → (S(ψ, φ) → S(ψ, χ)))`
`right_mono_until`	BX Temporal	φ, ψ, χ	Base	`((U(((φ → ψ) → ⊥), (⊥ → ⊥)) → ⊥) → (U(φ, χ) → U(ψ, χ)))`
`right_mono_since`	BX Temporal	φ, ψ, χ	Base	`((S(((φ → ψ) → ⊥), (⊥ → ⊥)) → ⊥) → (S(φ, χ) → S(ψ, χ)))`
`connect_future`	BX Temporal	φ	Base	`(φ → (U((S(φ, (⊥ → ⊥)) → ⊥), (⊥ → ⊥)) → ⊥))`
`connect_past`	BX Temporal	φ	Base	`(φ → (S((U(φ, (⊥ → ⊥)) → ⊥), (⊥ → ⊥)) → ⊥))`
`enrichment_until`	BX Temporal	φ, ψ, p	Base	`(((p → (U(ψ, φ) → ⊥)) → ⊥) → U(((ψ → (S(p, φ) → ⊥)) → ⊥), φ))`
`enrichment_since`	BX Temporal	φ, ψ, p	Base	`(((p → (S(ψ, φ) → ⊥)) → ⊥) → S(((ψ →`

Name	Layer	Params	Class	Schema (primitive basis)
				(U(p, φ) → ⊥)) → ⊥), φ))
self_accum_until	BX Temporal	φ, ψ	Base	(U(ψ, φ) → U(ψ, ((φ → (U(ψ, φ) → ⊥)) → ⊥)))
self_accum_since	BX Temporal	φ, ψ	Base	(S(ψ, φ) → S(ψ, ((φ → (S(ψ, φ) → ⊥)) → ⊥)))
absorb_until	BX Temporal	φ, ψ	Base	(U(((φ → (U(ψ, φ) → ⊥)) → ⊥), φ) → U(ψ, φ))
absorb_since	BX Temporal	φ, ψ	Base	(S(((φ → (S(ψ, φ) → ⊥)) → ⊥), φ) → S(ψ, φ))
linear_until	BX Temporal	φ, ψ, χ, ϑ	Base	(((U(ψ, φ) → (U(θ, χ) → ⊥)) → ⊥) → ((((U(((ψ → (θ → ⊥)) → ⊥), ((φ → (χ → ⊥)) → ⊥)) → ⊥) → U(((ψ → (χ → ⊥)) → ⊥), ((φ → (χ → ⊥)) → ⊥))) → ⊥) → U(((φ → (θ → ⊥)) → ⊥), ((φ → (χ → ⊥)) → ⊥))))
linear_since	BX Temporal	φ, ψ, χ, ϑ	Base	(((S(ψ, φ) → (S(θ, χ) → ⊥)) → ⊥) → ((((S(((ψ → (θ → ⊥)) → ⊥), ((φ → (χ → ⊥)) → ⊥)) → ⊥) → S(((ψ → (χ → ⊥)) → ⊥), ((φ → (χ → ⊥)) → ⊥))) → ⊥) → S(((φ → (θ → ⊥)) → ⊥), ((φ → (χ → ⊥)) → ⊥))))
until_F	BX Temporal	φ, ψ	Base	(U(ψ, φ) → U(ψ, (⊥ → ⊥)))
since_P	BX Temporal	φ, ψ	Base	(S(ψ, φ) → S(ψ, (⊥ → ⊥)))
temp_linearity	Additional BX Temporal	φ, ψ	Base	(((U(φ, (⊥ → ⊥)) → (U(ψ, (⊥ → ⊥)) → ⊥)) → ⊥) → ((U(((φ → (ψ → ⊥)) → ⊥),

Name	Layer	Params	Class	Schema (primitive basis)
				(⊥ → ⊥)) → ⊥) → ((U(((φ → (U(ψ, (⊥ → ⊥)) → ⊥)) → ⊥), (⊥ → ⊥)) → ⊥) → U(((U(φ, (⊥ → ⊥)) → (ψ → ⊥)) → ⊥), (⊥ → ⊥))))
temp_linearity_past	Additional BX Temporal	φ, ψ	Base	(((S(φ, (⊥ → ⊥)) → (S(ψ, (⊥ → ⊥)) → ⊥)) → ⊥) → ((S(((φ → ψ → ⊥)) → ⊥), (⊥ → ⊥)) → ⊥) → ((S(((φ → (S(ψ, (⊥ → ⊥)) → ⊥)) → ⊥), (⊥ → ⊥)) → ⊥) → S(((S(φ, (⊥ → ⊥)) → (ψ → ⊥)) → ⊥), (⊥ → ⊥)))))
F_until_equiv	Additional BX Temporal	φ	Base	(U(φ, (⊥ → ⊥)) → U(φ, (⊥ → ⊥)))
P_since_equiv	Additional BX Temporal	φ	Base	(S(φ, (⊥ → ⊥)) → S(φ, (⊥ → ⊥)))
modal_future	Modal-Temporal Interaction	φ	Base	(□φ → □(U((φ → ⊥), (⊥ → ⊥)) → ⊥))
discrete_symm_fwd	Uniformity		Base	(U((⊥ → ⊥), ⊥) → S((⊥ → ⊥), ⊥))
discrete_symm_bwd	Uniformity		Base	(S((⊥ → ⊥), ⊥) → U((⊥ → ⊥), ⊥))
discrete_propagate_fwd	Uniformity		Base	(U((⊥ → ⊥), ⊥) → (U((U((⊥ → ⊥), ⊥) → ⊥), (⊥ → ⊥)) → ⊥))
discrete_propagate_bwd	Uniformity		Base	(U((⊥ → ⊥), ⊥) → (S((U((⊥ → ⊥), ⊥) → ⊥), (⊥ → ⊥)) → ⊥))
discrete_box_necessity	Uniformity		Base	(U((⊥ → ⊥), ⊥) → □U((⊥ → ⊥), ⊥))
prior_UZ	Prior	φ	Discrete	(U(φ, (⊥ → ⊥)) → U(φ, (φ → ⊥)))
prior_SZ	Prior	φ	Discrete	(S(φ, (⊥ → ⊥)) → S(φ, (φ → ⊥)))
z1	Z1	φ	Discrete	((U((((U((φ → ⊥), (⊥ → ⊥)) → ⊥) → φ) → ⊥), (⊥ → ⊥)) → ⊥)

Name	Layer	Params	Class	Schema (primitive basis)
				`→ (U((U((φ → ⊥), (⊥ → ⊥)) → ⊥), (⊥ → ⊥)) → (U((φ → ⊥), (⊥ → ⊥)) → ⊥)))`
`density`	Density	φ	Dense	`((U(((U((φ → ⊥), (⊥ → ⊥)) → ⊥) → ⊥), (⊥ → ⊥)) → ⊥) → (U((φ → ⊥), (⊥ → ⊥)) → ⊥))`
`dense_indicator`	Density		Dense	`(U((⊥ → ⊥), ⊥) → ⊥)`

Table 28: The 42 axiom constructors of `Axiom`, in source order, with schemas extracted from the type index and frame classes from `Axiom.minFrameClass`.

The Inference Rules (7 constructors, `ProofSystem/Derivation.lean`)

Name	Premises	Conclusion	Side condition
`axiom`	—	$\Gamma \vdash_{[fc]} \varphi$	φ is an instance of an axiom schema with minFrameClass $\leq$ fc
`assumption`	—	$\Gamma \vdash_{[fc]} \varphi$	$\varphi \in \Gamma$
`modus_ponens`	$\Gamma \vdash_{[fc]} \varphi \to \psi$; $\Gamma \vdash_{[fc]} \varphi$	$\Gamma \vdash_{[fc]} \psi$	—
`necessitation`	$\vdash_{[fc]} \varphi$	$\vdash_{[fc]} \Box\varphi$	empty context only (theorems)
`temporal_necessitation`	$\vdash_{[fc]} \varphi$	$\vdash_{[fc]} G\varphi$	empty context only (theorems)
`temporal_duality`	$\vdash_{[fc]} \varphi$	$\vdash_{[fc]}$ swap_temporal φ	empty context only (theorems)
`weakening`	$\Gamma \vdash_{[fc]} \varphi$	$\Delta \vdash_{[fc]} \varphi$	$\Gamma \subseteq \Delta$

Table 29: The 7 inference rules (constructors of `DerivationTree`). The three necessitation-style rules apply to theorems only (empty context).

The Derived Operators (21 definitions, `Syntax/Formula.lean`)

Name	Params	Definition (primitive basis)
`top`		`(⊥ → ⊥)`
`neg`	φ	`(φ → ⊥)`
`some_future`	φ	`U(φ, (⊥ → ⊥))`
`some_past`	φ	`S(φ, (⊥ → ⊥))`
`all_future`	φ	`(U((φ → ⊥), (⊥ → ⊥)) → ⊥)`
`all_past`	φ	`(S((φ → ⊥), (⊥ → ⊥)) → ⊥)`

Name	Params	Definition (primitive basis)
and	φ, ψ	((φ → (ψ → ⊥)) → ⊥)
or	φ, ψ	((φ → ⊥) → ψ)
diamond	φ	(□(φ → ⊥) → ⊥)
always	φ	(((S((φ → ⊥), (⊥ → ⊥)) → ⊥) → (((φ → ((U((φ → ⊥), (⊥ → ⊥)) → ⊥) → ⊥)) → ⊥) → ⊥)) → ⊥)
next	φ	U(φ, ⊥)
prev	φ	S(φ, ⊥)
weak_future	φ	((φ → ((U((φ → ⊥), (⊥ → ⊥)) → ⊥) → ⊥)) → ⊥)
weak_past	φ	((φ → ((S((φ → ⊥), (⊥ → ⊥)) → ⊥) → ⊥)) → ⊥)
release	φ, ψ	(U((φ → ⊥), (ψ → ⊥)) → ⊥)
weak_until	φ, ψ	((U(φ, ψ) → ⊥) → (U((ψ → ⊥), (⊥ → ⊥)) → ⊥))
trigger	φ, ψ	(S((φ → ⊥), (ψ → ⊥)) → ⊥)
weak_since	φ, ψ	((S(φ, ψ) → ⊥) → (S((ψ → ⊥), (⊥ → ⊥)) → ⊥))
strong_release	φ, ψ	U(((ψ → (φ → ⊥)) → ⊥), ψ)
strong_trigger	φ, ψ	S(((ψ → (φ → ⊥)) → ⊥), ψ)
sometimes	φ	((((S(((φ → ⊥) → ⊥), (⊥ → ⊥)) → ⊥) → ((((φ → ⊥) → ((U(((φ → ⊥) → ⊥), (⊥ → ⊥)) → ⊥) → ⊥)) → ⊥) → ⊥)) → ⊥) → ⊥)

Table 30: The 21 derived operators, each unfolded to the primitive basis by the Lean kernel (the real `def` applied to schematic atoms).

Generated from live source at commit `a25f7d4e3` *(2026-07-07) — never hand-copied. Regenerate:* `scripts/typst-machine-appendix.sh`.

References

Bibliography

[1] B. Brast-McKie, “The Construction of Possible Worlds,” *Journal of Philosophical Logic*, 2026, [Online]. Available: https://benbrastmckie.com/wp-content/uploads/2026/07/possible_worlds.pdf

[2] J. P. Burgess, “Axioms for Tense Logic I: "Since" and "Until",” *Notre Dame Journal of Formal Logic*, vol. 23, no. 4, pp. 367–374, 1982.

[3] J. P. Burgess, “Basic Tense Logic,” *Handbook of Philosophical Logic, Volume II: Extensions of Classical Logic*. D. Reidel, pp. 89–133, 1984.

[4] M. Xu, “Until and Since: Completeness and Decidability,” 1988.

[5] M. Reynolds, “An Axiomatization for Until and Since over the Reals Without the IRR Rule,” *Studia Logica*, vol. 51, 1992.

[6] K. Doets, “Completeness and Definability: Applications of the Ehrenfeucht Game in Second-Order and Intensional Logic,” Doctoral dissertation, 1987.

[7] H. Kamp, “Formal Properties of "Now",” *Theoria*, vol. 37, no. 3, pp. 227–273, 1971.

[8] A. Pnueli, “The Temporal Logic of Programs,” in *18th Annual Symposium on Foundations of Computer Science (SFCS 1977)*, 1977, pp. 46–57.

[9] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[10] S. Demri, V. Goranko, and M. Lange, *Temporal Logics in Computer Science: Finite-State Systems*. Cambridge University Press, 2016.

[11] D. M. Gabbay, A. Kurucz, F. Wolter, and M. Zakharyaschev, *Many-Dimensional Modal Logics: Theory and Applications*, vol. 148. in Studies in Logic and the Foundations of Mathematics, vol. 148. Elsevier, 2003.

[12] E. M. Clarke and E. A. Emerson, “Design and Synthesis of Synchronization Skeletons Using Branching Time Temporal Logic,” in *Logics of Programs*, D. Kozen, Ed., Berlin, Heidelberg: Springer, 1982, pp. 52–71.

[13] E. A. Emerson and J. Y. Halpern, “"Sometimes" and "Not Never" Revisited: On Branching versus Linear Time Temporal Logic,” *Journal of the ACM*, vol. 33, no. 1, pp. 151–178, 1986.

[14] L. Lamport, “"Sometime" Is Sometimes "Not Never": On the Temporal Logic of Programs,” in *Proceedings of the 7th ACM SIGPLAN-SIGACT Symposium on Principles of Programming*

Languages (POPL '80), New York: Association for Computing Machinery, 1980, pp. 174–185.

[15] M. Y. Vardi, "Branching vs. Linear Time: Final Showdown," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2001)*, T. Margaria and W. Yi, Eds., Berlin, Heidelberg: Springer, 2001, pp. 1–22.

[16] M. R. Clarkson, B. Finkbeiner, M. Koleini, K. K. Micinski, M. N. Rabe, and C. Sánchez, "Temporal Logics for Hyperproperties," in *Principles of Security and Trust (POST)*, in LNCS, vol. 8414. Springer, 2014, pp. 265–284.

[17] B. Finkbeiner and C. Hahn, "Deciding Hyperproperties," in *27th International Conference on Concurrency Theory (CONCUR 2016)*, in LIPIcs, vol. 59. 2016, pp. 13:1–13:14.

[18] N. Belnap, M. Perloff, and M. Xu, *Facing the Future: Agents and Choices in Our Indeterminist World*. Oxford: Oxford University Press, 2001.

[19] A. Rumberg, "Transition Semantics for Branching Time," *Journal of Logic, Language and Information*, vol. 25, no. 1, pp. 77–108, 2016.

[20] R. H. Thomason, "Combinations of Tense and Modality," *Handbook of Philosophical Logic, Volume II: Extensions of Classical Logic*. Springer Netherlands, Dordrecht, pp. 135–165, 1984.

[21] D. Lind and B. Marcus, *An Introduction to Symbolic Dynamics and Coding*, 2nd ed. Cambridge: Cambridge University Press, 2021.

[22] F. Vlach, ""Now" and "Then": A Formal Study in the Logic of Tense Anaphora," Doctoral dissertation, 1973.

[23] M. J. Cresswell, *Entities and Indices*, vol. 41. in Studies in Linguistics and Philosophy, vol. 41. Dordrecht: Kluwer Academic Publishers, 1990.

[24] P. Blackburn, "Representation, Reasoning, and Relational Structures: a Hybrid Logic Manifesto," *Logic Journal of the IGPL*, vol. 8, no. 3, pp. 339–365, 2000.

[25] V. Goranko, "Hierarchies of Modal and Temporal Logics with Reference Pointers," *Journal of Logic, Language and Information*, vol. 5, no. 1, pp. 1–24, 1996.

[26] J. A. W. Kamp, "Tense Logic and the Theory of Linear Order," Doctoral dissertation, 1968.

[27] A. Rabinovich, "A Proof of Kamp's Theorem," *Logical Methods in Computer Science*, vol. 10, no. 1, 2014.

[28] D. M. Gabbay, A. Pnueli, S. Shelah, and J. Stavi, "On the Temporal Analysis of Fairness," in *Proceedings of the 7th ACM Symposium on Principles of Programming Languages (POPL)*, 1980, pp. 163–173.

[29] A. P. Sistla and E. M. Clarke, "The Complexity of Propositional Linear Temporal Logics," *Journal of the ACM*, vol. 32, no. 3, pp. 733–749, 1985.

[30] M. Marx, "Complexity of Products of Modal Logics," *Journal of Logic and Computation*, vol. 9, no. 2, pp. 197–214, 1999.

[31] J. Y. Halpern, R. van der Meyden, and M. Y. Vardi, "Complete Axiomatizations for Reasoning about Knowledge and Time," *SIAM Journal on Computing*, vol. 33, no. 3, pp. 674–703, 2004.

[32] R. Hirsch, I. Hodkinson, and A. Kurucz, "On Modal Logics Between K × K × K and S5 × S5 × S5," *Journal of Symbolic Logic*, vol. 67, no. 1, pp. 221–234, 2002.

[33] C. Areces, P. Blackburn, and M. Marx, "Hybrid Logics: Characterization, Interpolation and Complexity," *Journal of Symbolic Logic*, vol. 66, no. 3, pp. 977–1010, 2001.

[34] B. ten Cate and M. Franceschet, "On the Complexity of Hybrid Logics with Binders," in *Proceedings of the 19th International Workshop on Computer Science Logic (CSL)*, in LNCS, vol. 3634. 2005, pp. 339–354.

[35] M. Franceschet, M. de Rijke, and B.-H. Schlingloff, "Hybrid Logics on Linear Structures: Expressivity and Complexity," in *Proceedings of the 10th International Symposium on Temporal Representation and Reasoning (TIME-ICTL)*, 2003, pp. 166–173.

[36] S. Demri and R. Lazić, "LTL with the Freeze Quantifier and Register Automata," *ACM Transactions on Computational Logic*, vol. 10, no. 3, 2009.

[37] R. Alur and T. A. Henzinger, "A Really Temporal Logic," *Journal of the ACM*, vol. 41, no. 1, pp. 181–204, 1994.

[38] B. Finkbeiner, M. N. Rabe, and C. Sánchez, "Algorithms for Model Checking HyperLTL and HyperCTL*," in *Computer Aided Verification (CAV)*, in LNCS, vol. 9206. Springer, 2015, pp. 30–48.

[39] B. Finkbeiner and M. Zimmermann, "The First-Order Logic of Hyperproperties," in *34th Symposium on Theoretical Aspects of Computer Science (STACS 2017)*, in LIPIcs, vol. 66. 2017, pp. 30:1–30:14.

[40] D. M. Gabbay, I. Hodkinson, and M. Reynolds, *Temporal Logic: Mathematical Foundations and Computational Aspects*. Oxford University Press, 1994.

[41] P. Blackburn, M. de Rijke, and Y. Venema, *Modal Logic*. Cambridge University Press, 2001.